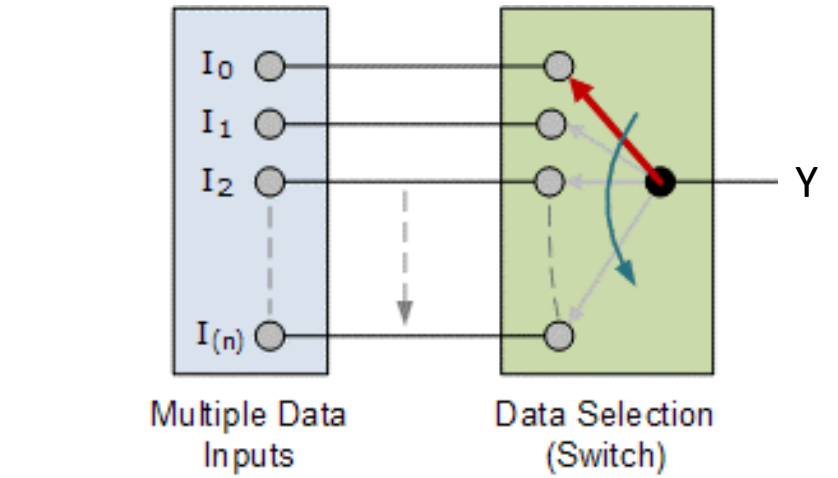
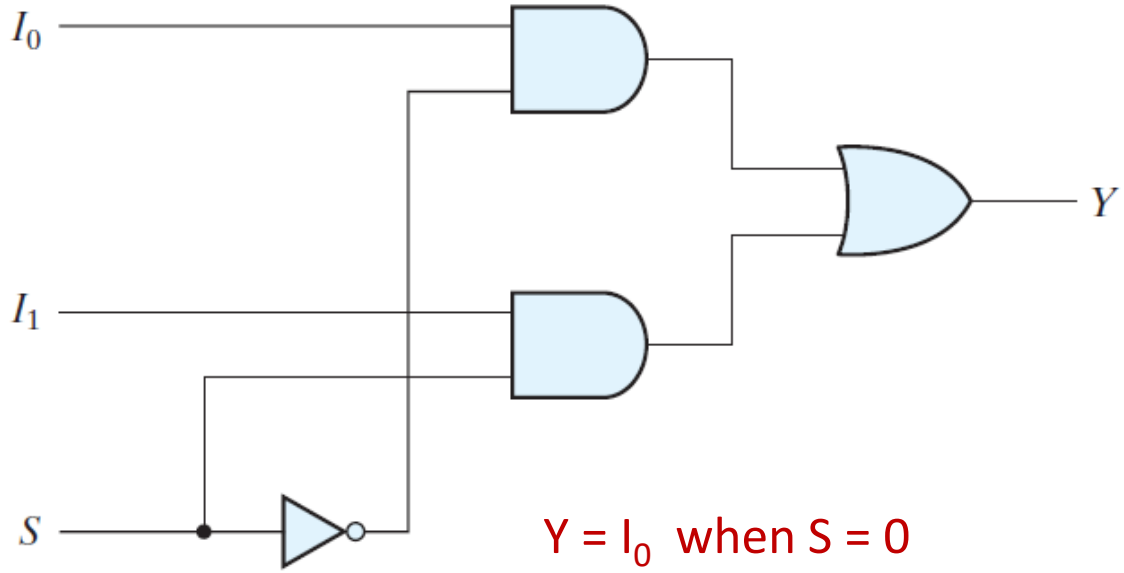
Lecture 12 – Combinational logic circuits

Multiplexer

- *A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line*
- The selection of a particular input line is controlled by a set of *selection lines*
- Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected
  - Eg: A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination
- The multiplexer acts like an electronic switch that selects one of the sources



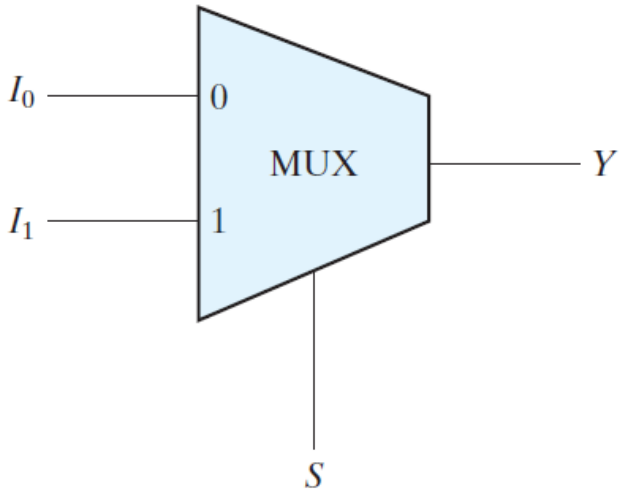


$Y = I_0$ when $S = 0$

$Y = I_1$ when $S = 1$

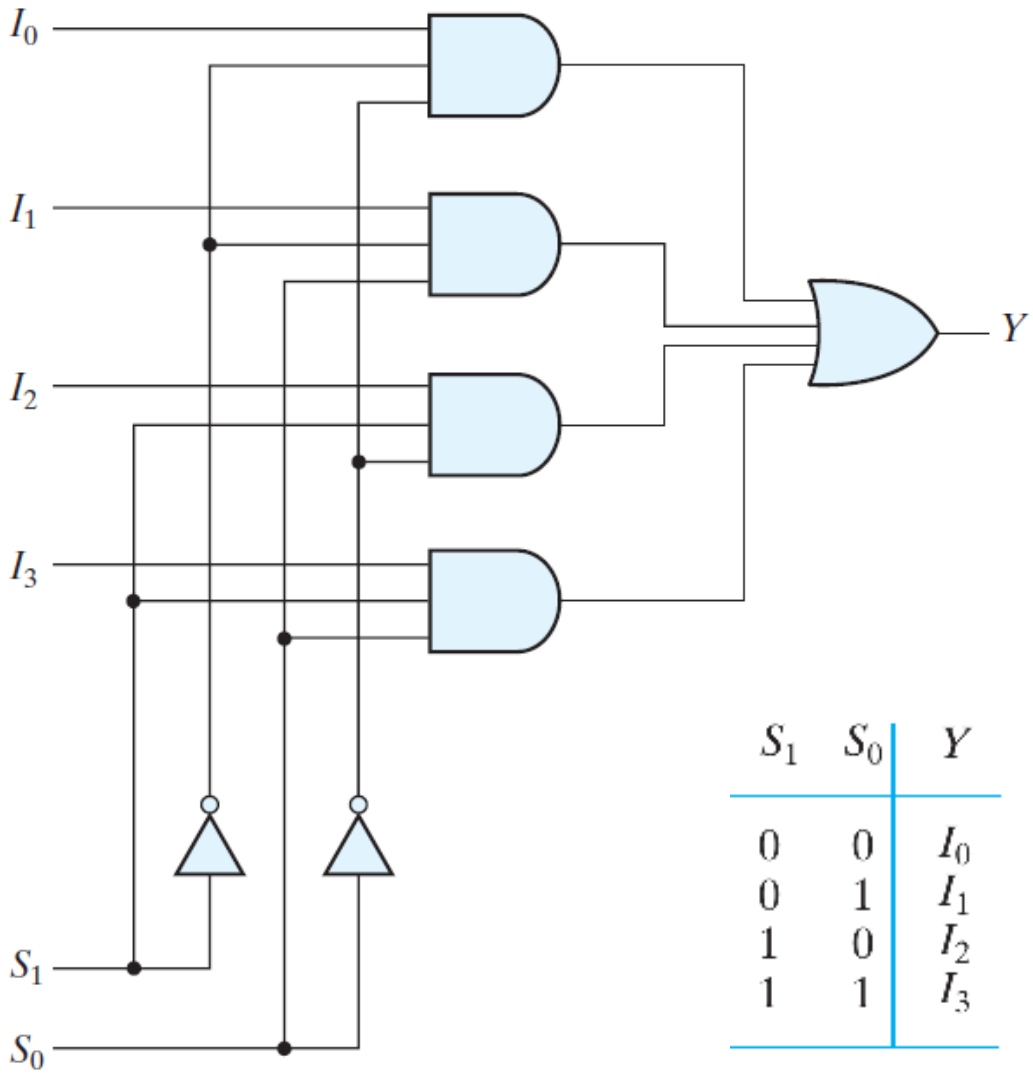
Multiplexer

The block diagram of a multiplexer (also called MUX) is sometimes depicted by a wedge-shaped symbol



Multiplexer

- Similarly, we can make a four-to-one-line multiplexer
- Each of the four inputs, I_0 through I_3, is applied to one input of an AND gate
- The outputs of the AND gates are applied to a single OR gate that provides the one-line output
- A multiplexer is also called a *data selector*, since it selects one of many inputs and steers the binary information to the output line
- *In general, a 2^n-to-1-line multiplexer is constructed ANDing each input line with 2^n minterms*
- The outputs of the AND gates are applied to a single OR gate

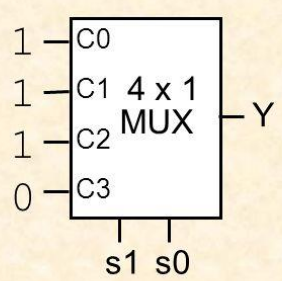


S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Multiplexer

- *We can use MUXes to implement Boolean functions*
- The minterms of a function are generated in a multiplexer by the circuit associated with the selection inputs
- The individual minterms can be selected by the data inputs, thereby providing a method of implementing a Boolean function of *n* variables with a multiplexer that has *n* selection inputs and 2^n data inputs, one for each minterm
- Example: implement NAND

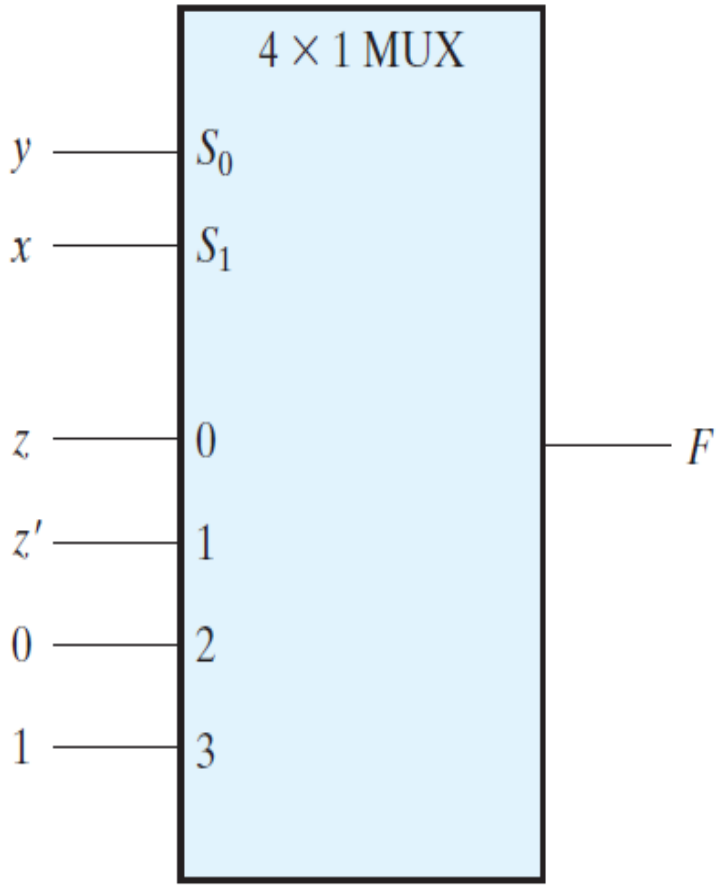
NAND implementation using MUX



s1	s0	Y = NAND	
0	0	C0	1
0	1	C1	1
1	0	C2	1
1	1	C3	0

Multiplexer

- There is a neat trick to obtain a more efficient method for implementing a Boolean function of *n* variables with a multiplexer that has *n* - 1 selection inputs
- The first *n* - 1 variables of the function are connected to the selection inputs of the multiplexer
- The remaining single variable of the function is used for the data inputs
- Say we have a three variable function F(x,y,z)
- We take a 4to1 MUX (with two input select lines) and each data input of the multiplexer will be *z*, *z'*, 1, or 0



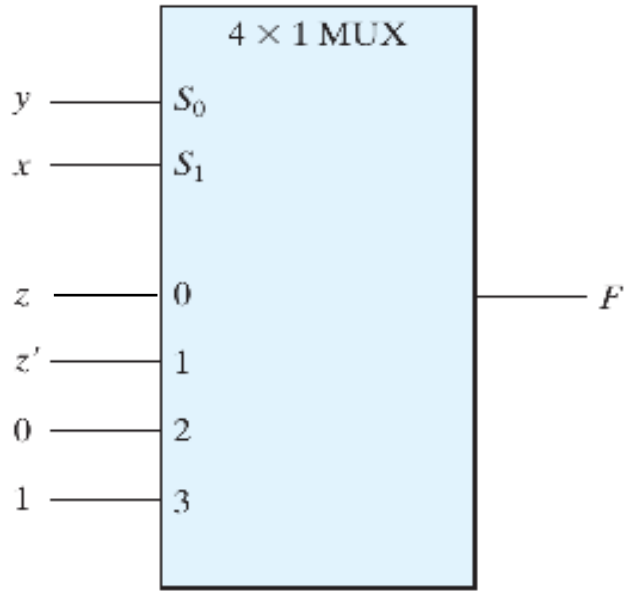
Multiplexer

- Consider the function:

$$F(x, y, z) = \sum(1,2,6,7)$$

- This function of three variables can be implemented with a four-to-one-line multiplexer
- The two variables *x* and *y* are applied to the selection lines in that order; *x* is connected to the S_1 input and *y* to the S_0 input
- The values for the data input lines are determined from the truth table of the function
- When *xy* = 00, output *F* is equal to *z* because *F* = 0 when *z* = 0 and *F* = 1 when *z* = 1
- This requires that variable *z* be applied to data input 0
- In a similar fashion, we can determine the required input to data lines 1, 2, and 3 from the value of *F* when *xy* = 01, 10, and 11, respectively

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	



Multiplexer

- The general procedure for implementing any Boolean function of *n* variables with a multiplexer with *n* - 1 selection inputs and 2^{n-1} data inputs follows from the previous example

1. To begin with, Boolean function is listed in a truth table
2. Then the most significant *n* - 1 variables in the table are applied to the selection inputs of the multiplexer
3. For each combination of the selection variables, we evaluate the output as a function of the last variable
4. This function can be 0, 1, the variable, or the complement of the variable
5. These values are then applied to the data inputs in the proper order

$$F(A, B, C, D) = \sum (1, 3, 4, 11, 12, 13, 14, 15)$$

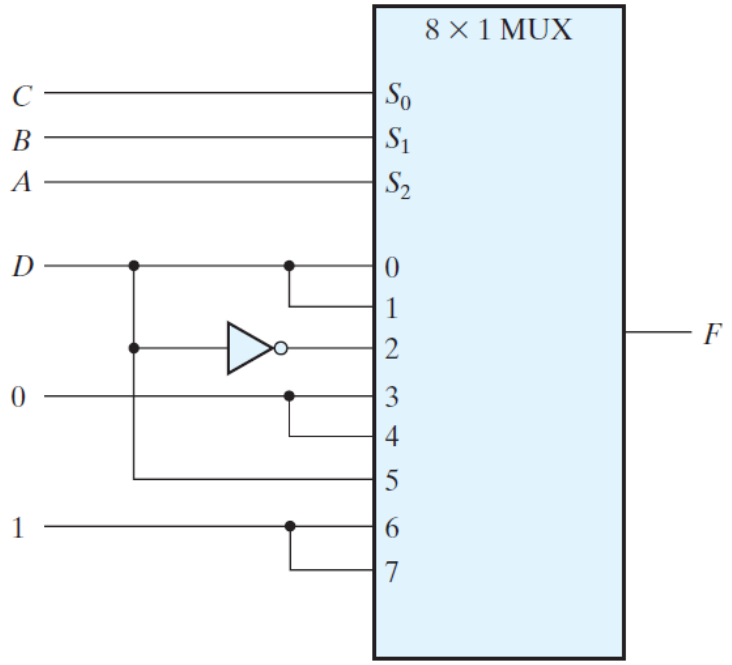
Multiplexer

- The general procedure for implementing any Boolean function of *n* variables with a multiplexer with *n* - 1 selection inputs and 2^{n-1} data inputs follows from the previous example

$$F(A, B, C, D) = \sum (1, 3, 4, 11, 12, 13, 14, 15)$$

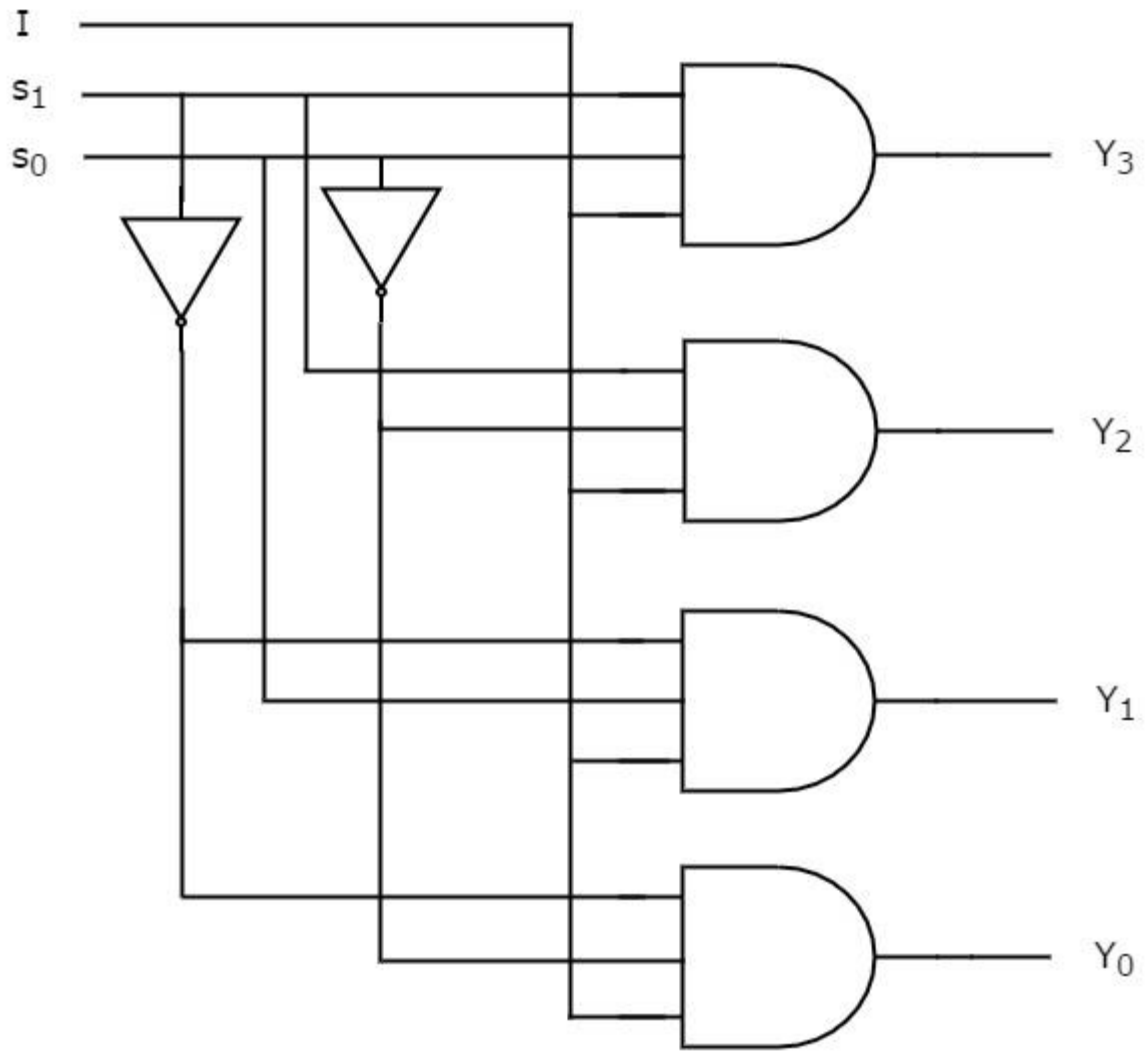
1. To begin with, Boolean function is listed in a truth table
2. Then the most significant *n* - 1 variables in the table are applied to the selection inputs of the multiplexer
3. For each combination of the selection variables, we evaluate the output as a function of the last variable
4. This function can be 0, 1, the variable, or the complement of the variable
5. These values are then applied to the data inputs in the proper order

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



Demultiplexer

- *Demultiplexers (Demux) do the exact opposite of MUX operation – take a single line input and direct it to an output line depending on the select line input*
- 1-to-2^n Demux will have n select lines
- We again make minterms from the available inputs and AND it with the single data line
- Based on the input signals the particular output is connected to the data line and all other outputs are zero

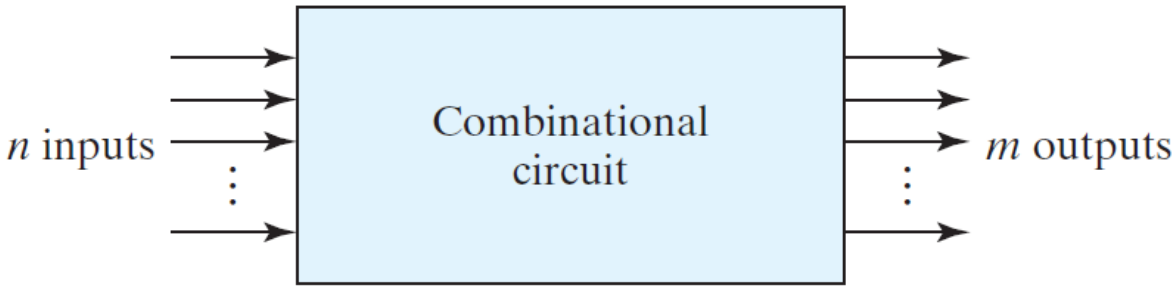


Combinational circuits

- Logic circuits for digital systems may be *combinational or sequential*
- A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs
- A combinational circuit performs an operation that can be specified logically by a set of Boolean functions
- *In contrast, sequential circuits employ storage elements in addition to logic gates*
- Their outputs are a function of the inputs and the state of the storage elements
- Because the state of the storage elements is a function of previous inputs, the outputs of a sequential circuit depend not only on present values of inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states

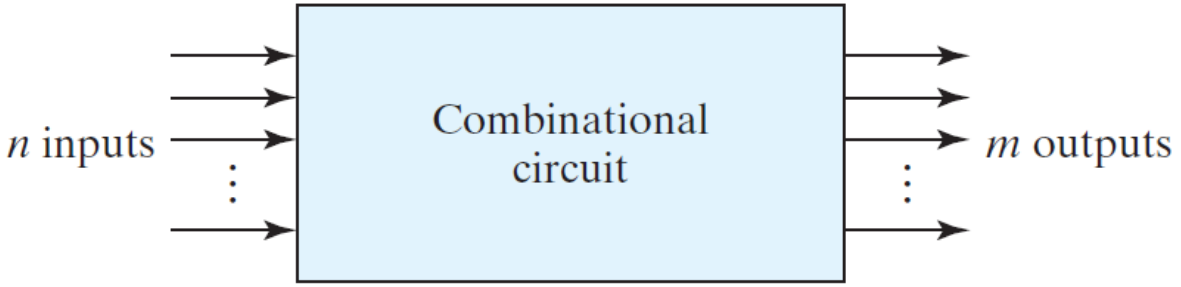
Combinational circuits

- A combinational circuit consists of an interconnection of logic gates
- Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data
- Each input and output variable exists physically as an analog signal whose values are interpreted to be a binary signal that represents logic 1 and logic 0



Combinational circuits

- For n input variables, there are 2^n possible combinations of the binary inputs
- For each possible input combination, there is one possible value for each output variable
- Thus, a combinational circuit can be specified with a truth table that lists the output values for each combination of input variables
- A combinational circuit also can be described by m Boolean functions, one for each output variable
- Each output function is expressed in terms of the n input variables



Circuit design

- The design of combinational circuits starts from the specification of the design objective and culminates in a logic circuit diagram or a set of Boolean functions from which the logic diagram can be obtained
- The procedure involves the following steps:

1. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each
2. Derive the truth table that defines the required relationship between inputs and outputs
3. Make the K-map, if necessary
4. Obtain the simplified Boolean functions for each output as a function of the input variables
5. Draw the logic diagram and verify the correctness of the design (manually or by simulation)

- Here we go…

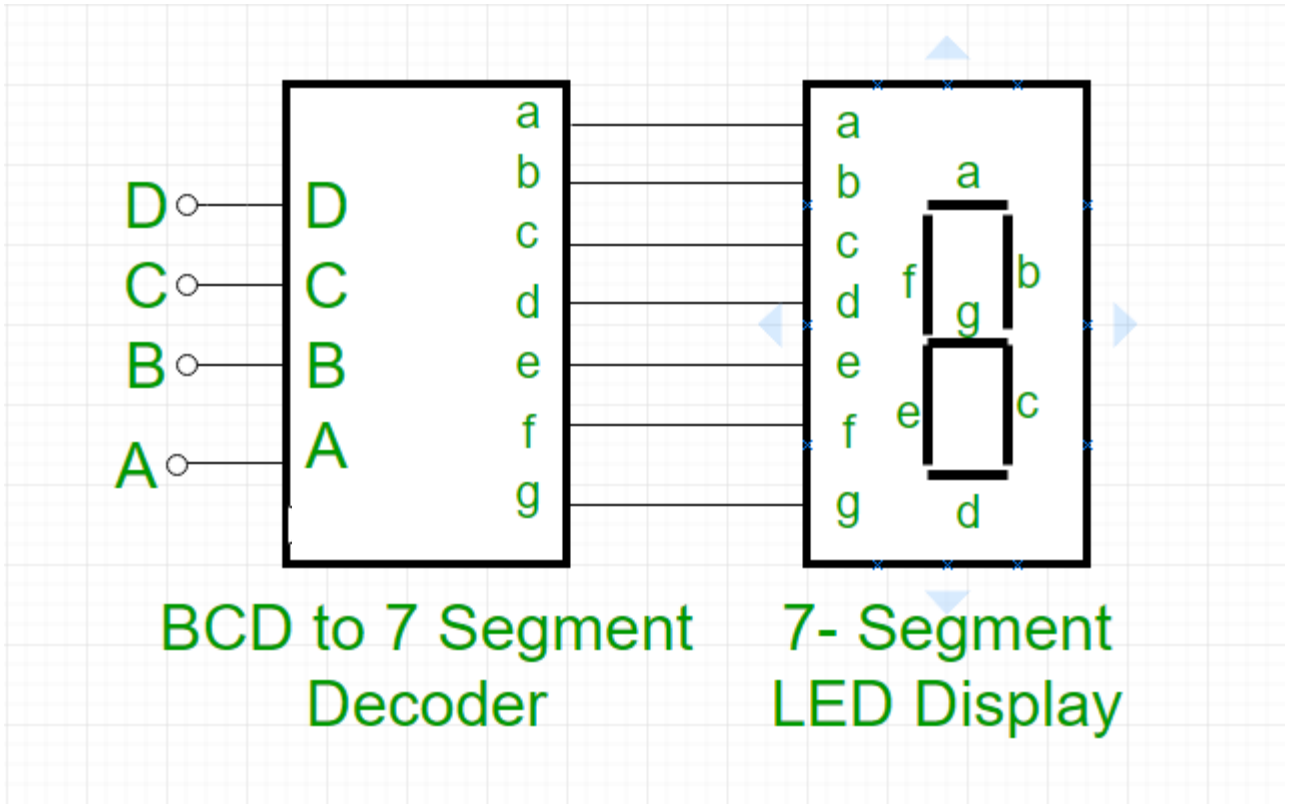
The 7-segment decoder



0 2 3 5 6 7 8 9

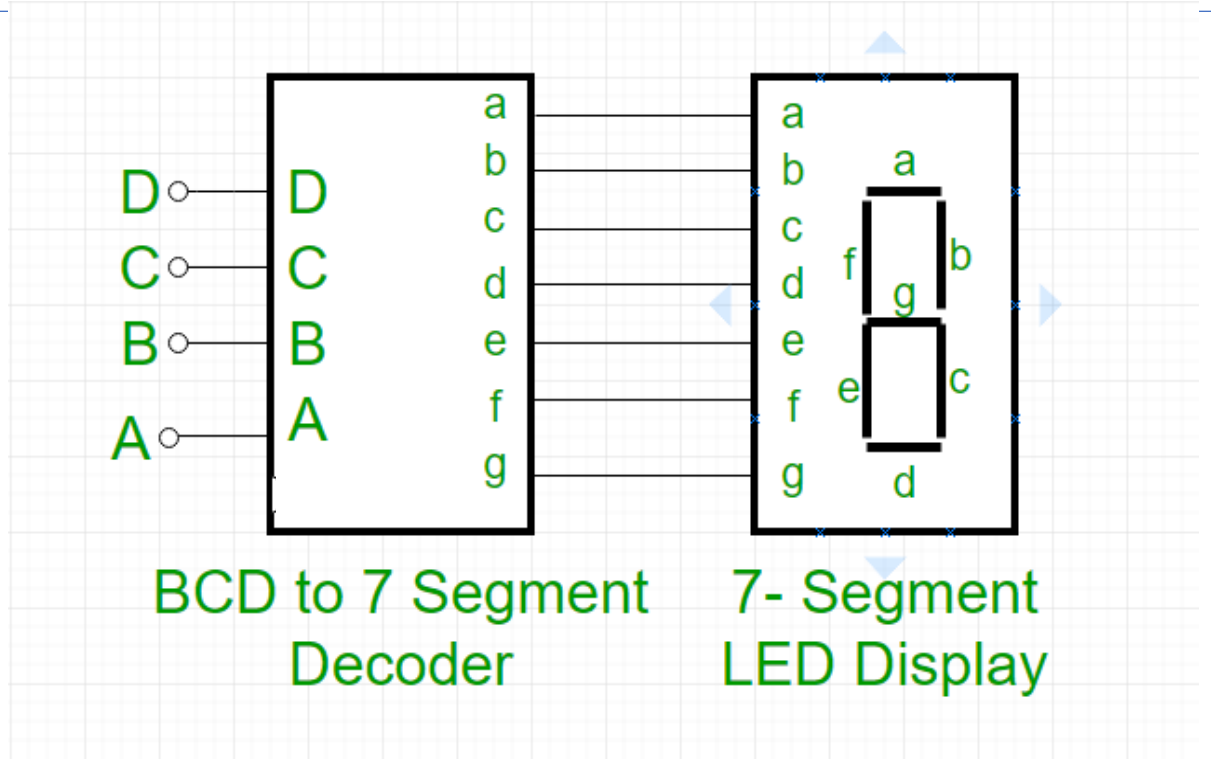
7-segment decoder

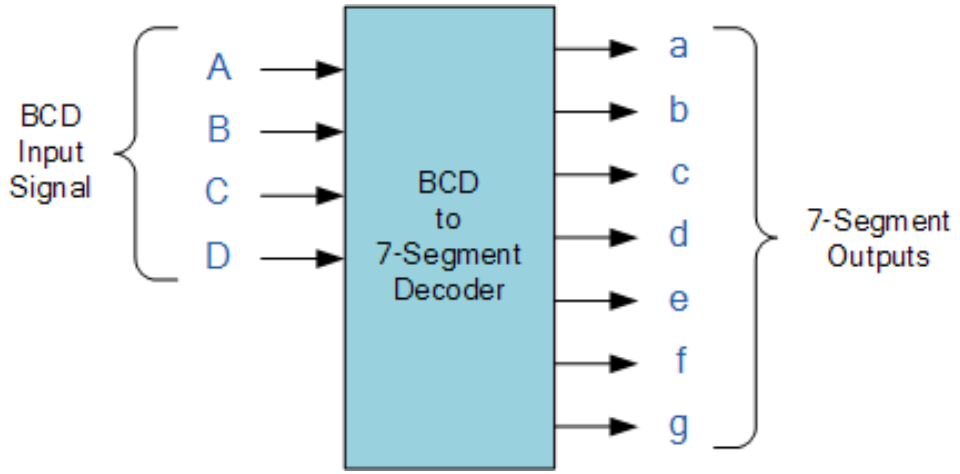
- We discussed many ways of representing symbols using binary variables – signed magnitude, 2's complement, BCD, ASCII etc.
- Going from one representation to the other may be considered as an encoding/decoding operation
- Consider the practical problem of displaying binary numbers using a 7-segment LED display
- For this, we need to "decode" the binary information into the display inputs



7-segment decoder

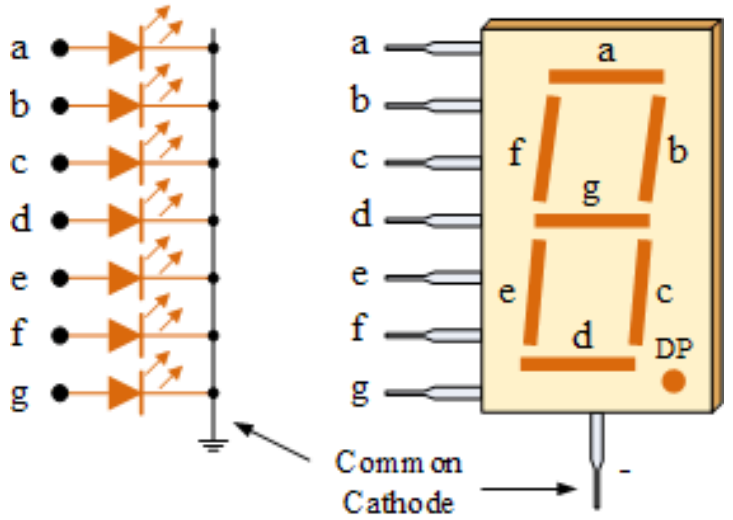
- First, we realize that there are 4 inputs and 7 outputs!
- Thus, the truth-table will be a 16 row table with 7 different columns for 7 outputs
- Hence, there will be 7 different functions we will be implementing to make this decoder
- The other thing we need to realize is whether a particular output should be HIGH or LOW for the LED to glow?

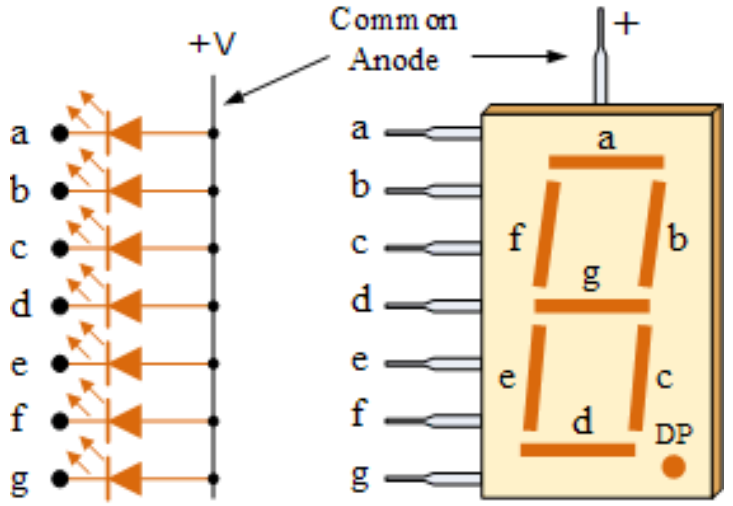




7-segment decoder

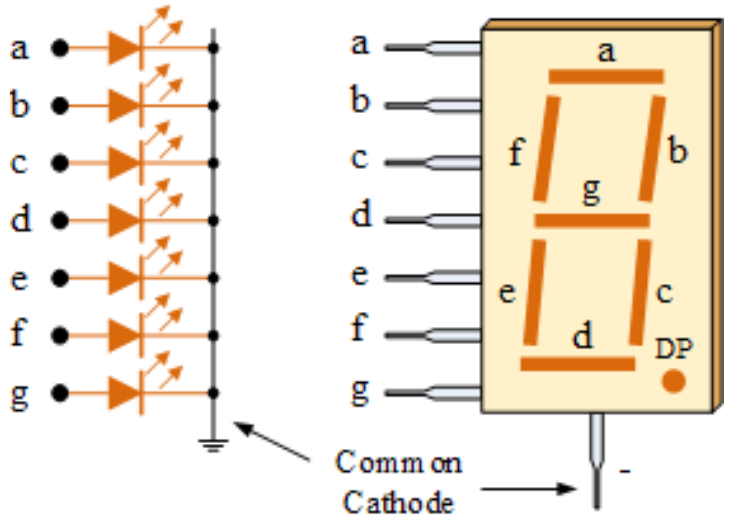
- The other thing we need to realize is whether a particular output should be HIGH or LOW for the LED to glow?
- To know the answer to this, we see that there are two types of 7-segment LED displays – common anode and common cathode
- The common cathode connects all LED cathodes to a common ground, thus, when input is high LED glows
- Conversely, the common anode connects all LED anodes to $+V_{cc}$, thus, when input is low, LED glows





7-segment decoder

- Let us choose common cathode LED display to make the function
- Thus, we can make the truth-table
- Obviously, the BCD system only goes from 0 to 9, while we have 16 rows for 4 inputs
- In the other rows, we fill all the outputs as *don't care*, because we are sure that these are not going to be input anyway (trust the engineer before you)
- Thus, we have six don't care conditions

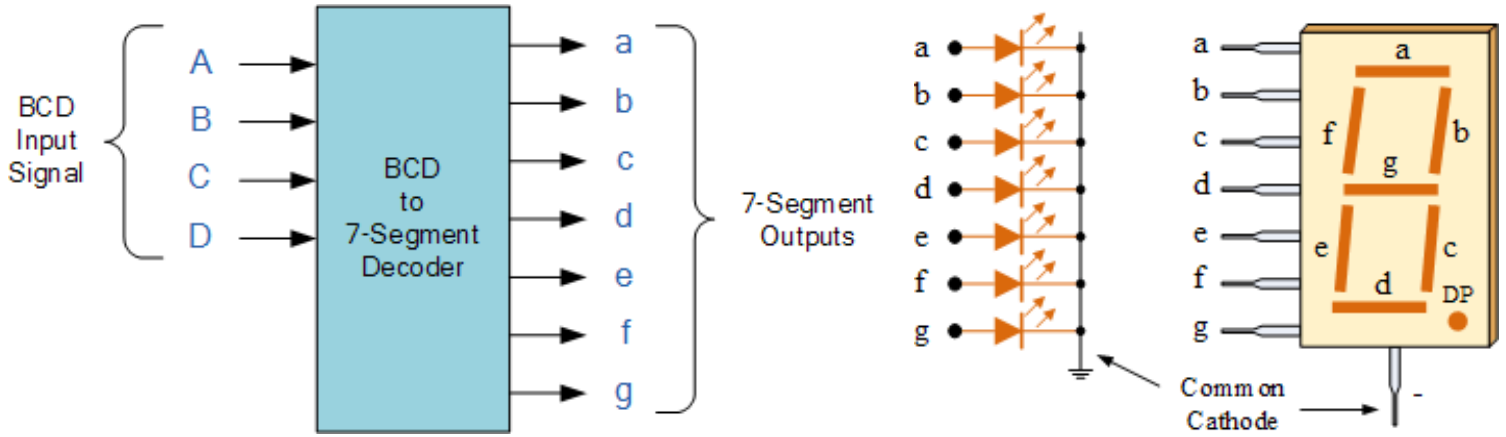


A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	1	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

Lecture 13 – Combinational logic circuits 2

7-segment decoder

- Let us choose common cathode LED display to make the function
- The BCD system only goes from 0 to 9, while we have 16 rows for 4 inputs
- In the other rows, we fill all the outputs as don't care, because we are sure that these are not going to be input anyway
- Thus, we have six don't care conditions



A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	1	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

7-segment decoder

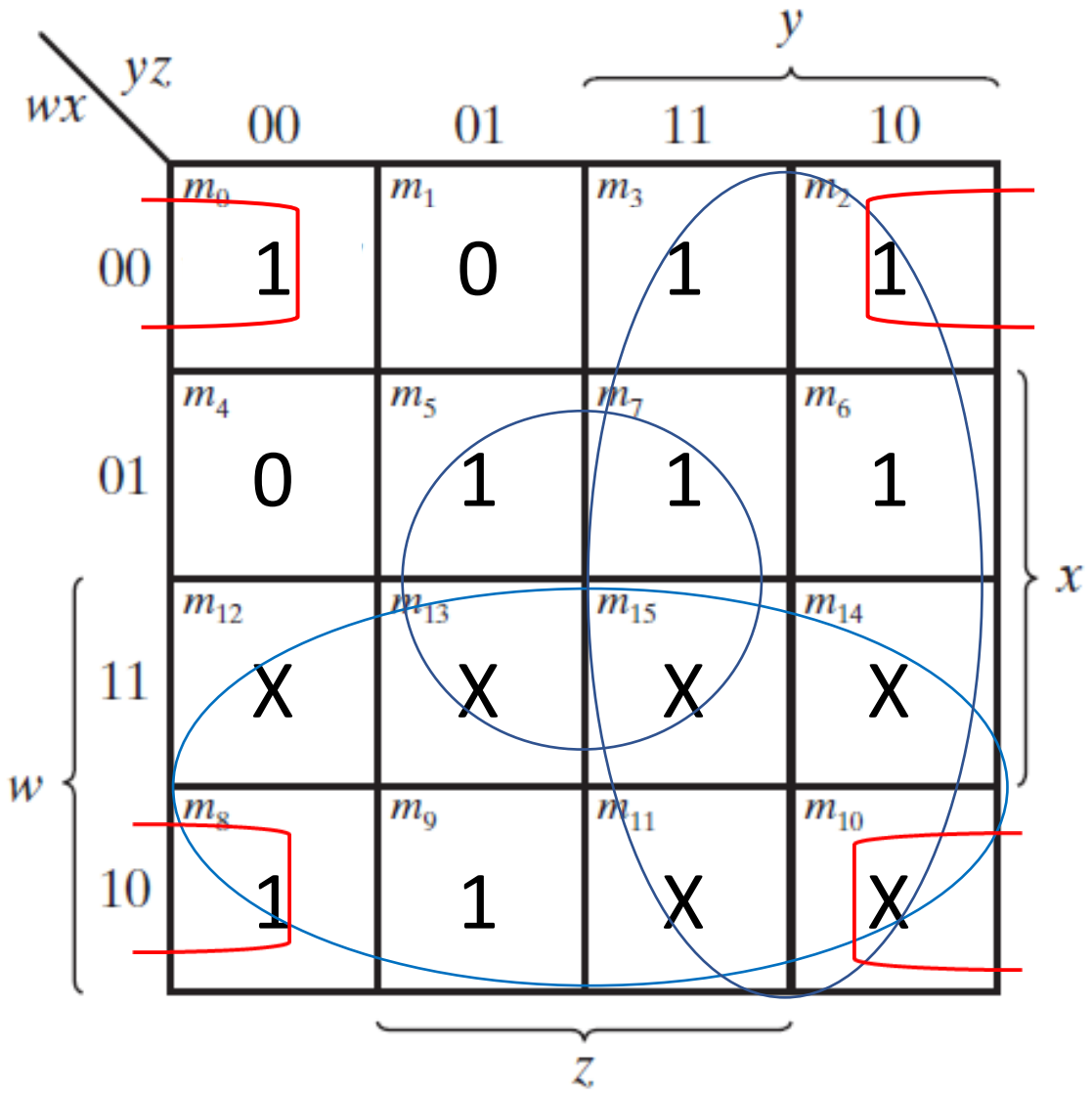
- Thus, the K-map for output a will look like this (A=w, B=x, C=y, D=z)
  - We have two clusters of 8: y and w
  - Two clusters of four: xz and $x'z'$
- Thus, the logic function for a can be:

$$F_a = y + w + xz + x'z'$$

- Or we can have PoS as:
  - One cluster of two: $xy'z'$
  - One min-term: $w'x'y'z$
- Thus,

$$F_a = (x' + y + z)(w + x + y + z')$$

- This can be done for all the other outputs

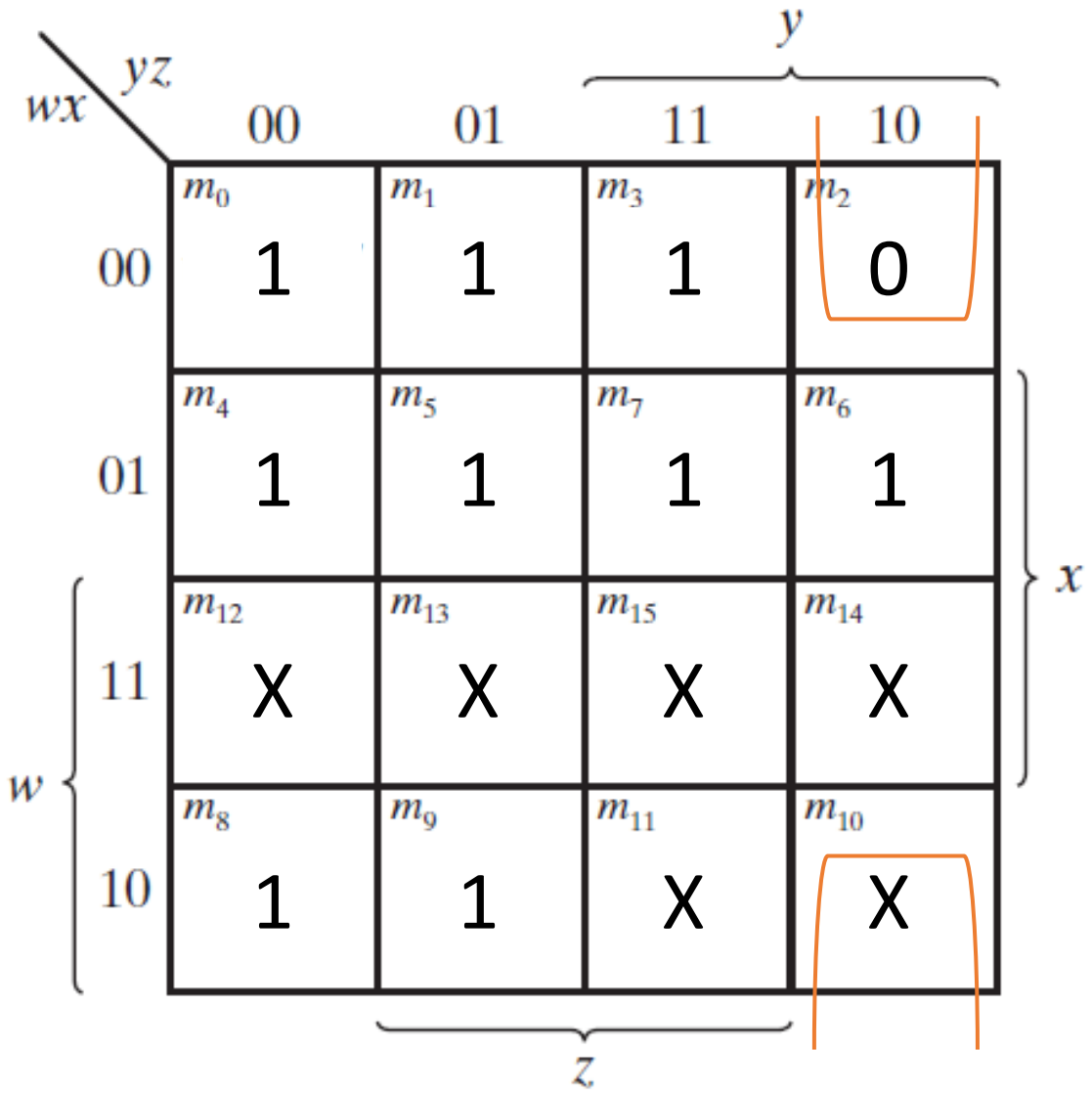


7-segment decoder

- Thus, the K-map for output c will look like this
- Consider PoS:
- The max term cluster of two is represented by yz'x'
- In PoS form:

$$c = y' + z + x$$

This can be done for all the other outputs



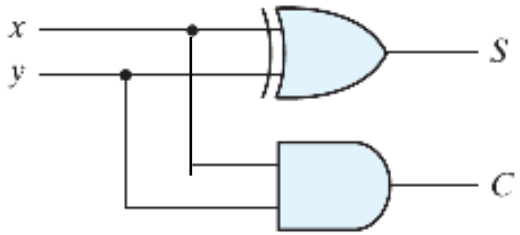
The Binary adder

Binary adder

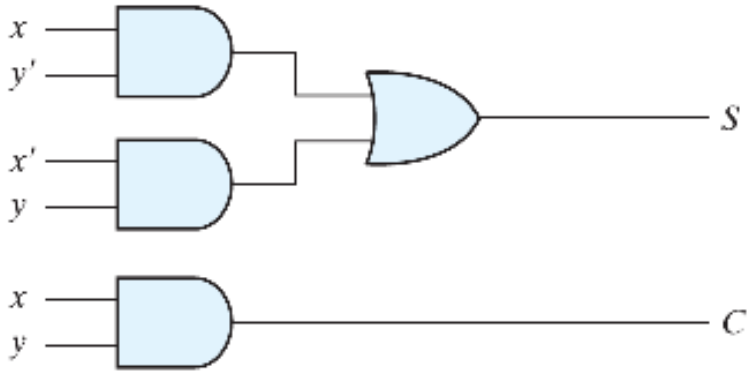
- Digital computers perform a variety of information-processing tasks
- Among the functions encountered are the various arithmetic operations
- The most basic arithmetic operation is the addition of two binary digits:
  - 0+0=0, 1+0=1, 0+1= 1, 1+1 = 10
- A combinational circuit that performs the addition of two bits is called a *half adder*

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



(b) $S = x \oplus y$
$C = xy$



(a) $S = xy' + x'y$
$C = xy$

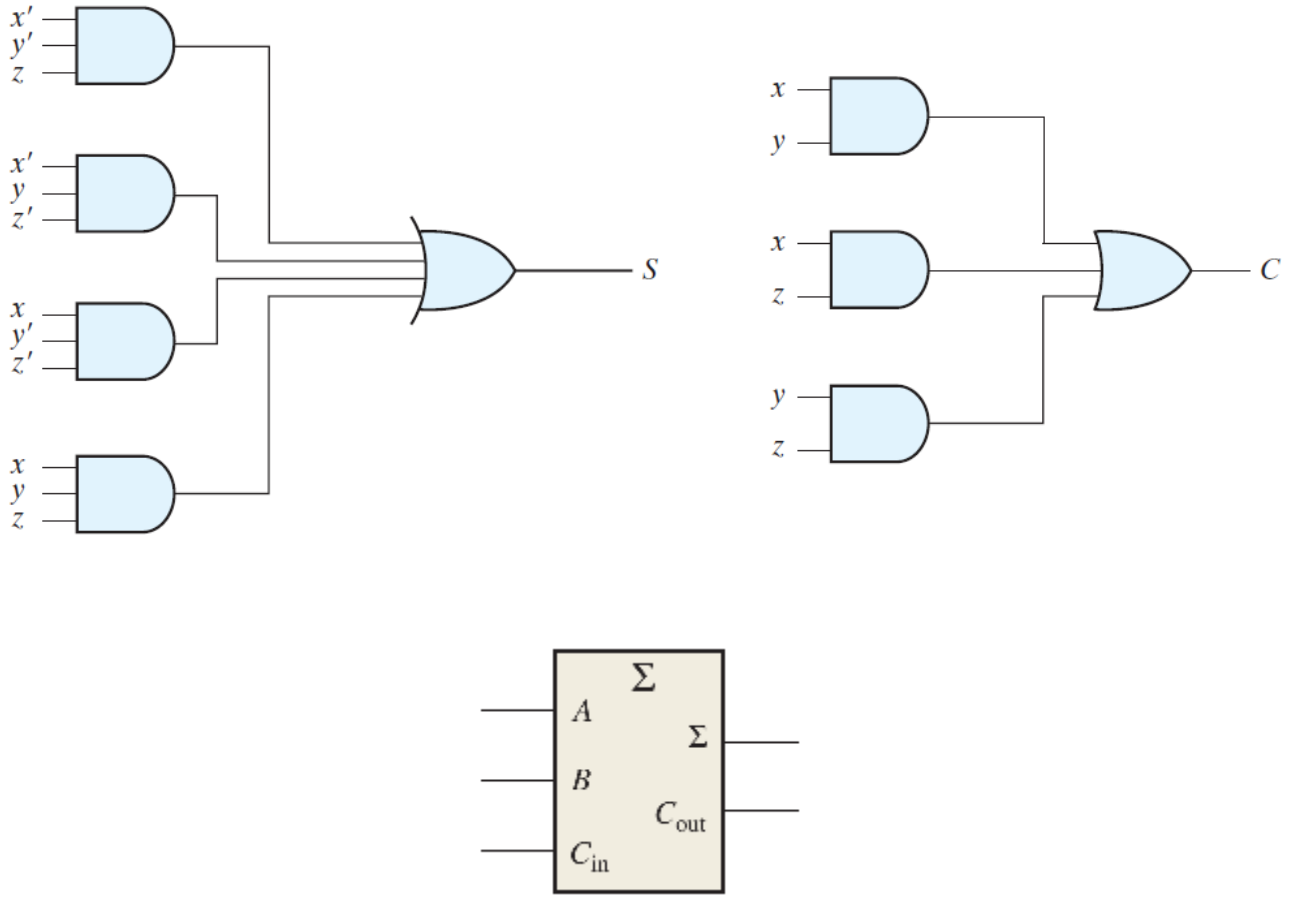
Sum S is also denoted by Σ

Binary adder

- Circuit that performs the addition of three bits (two significant bits and a previous carry) is a *full adder*

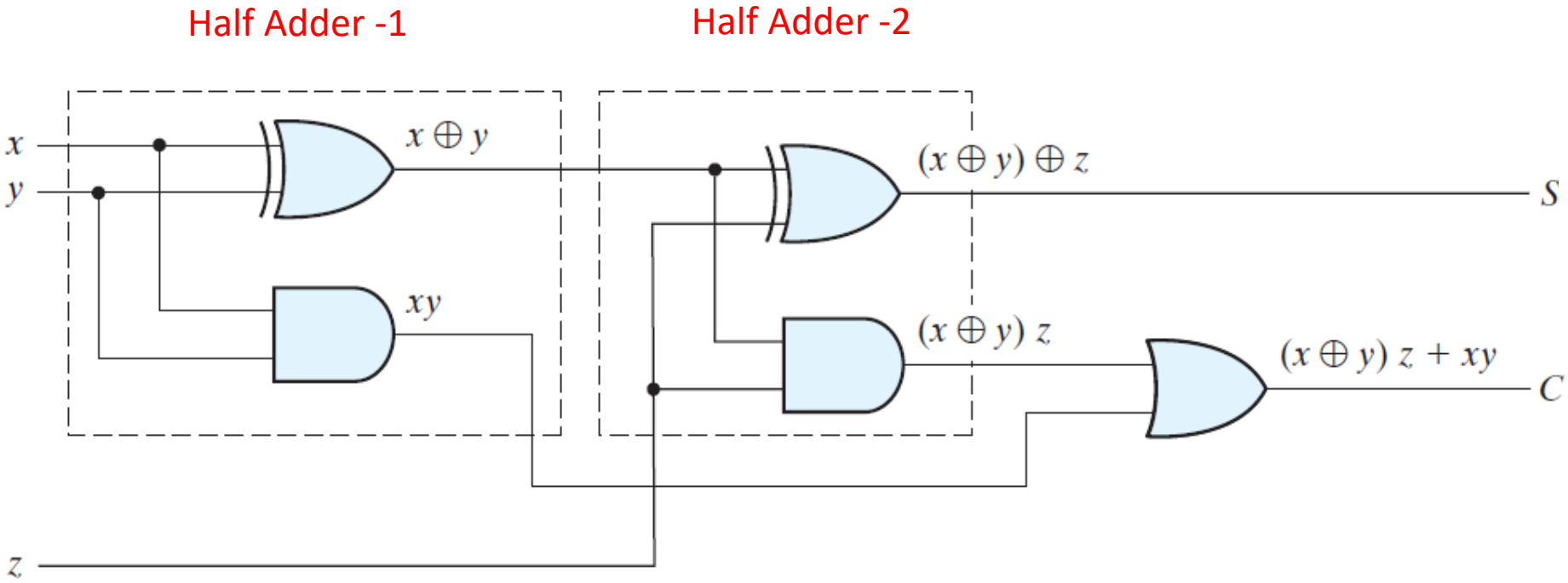
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Binary adder

- We can use two half adders to create a full adder



Full Adder

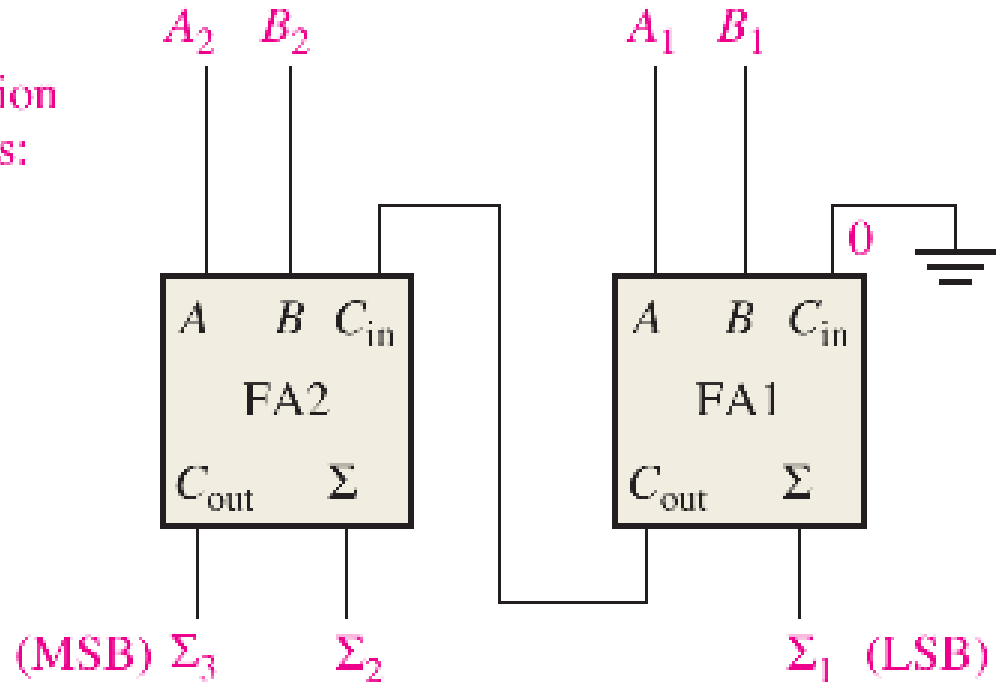
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

n-bit binary adder

- Addition of *n*-bit numbers requires a chain of *n* full adders or a chain of one-half adder and *n*-1 full adders
- Consider a 2-bit adder:

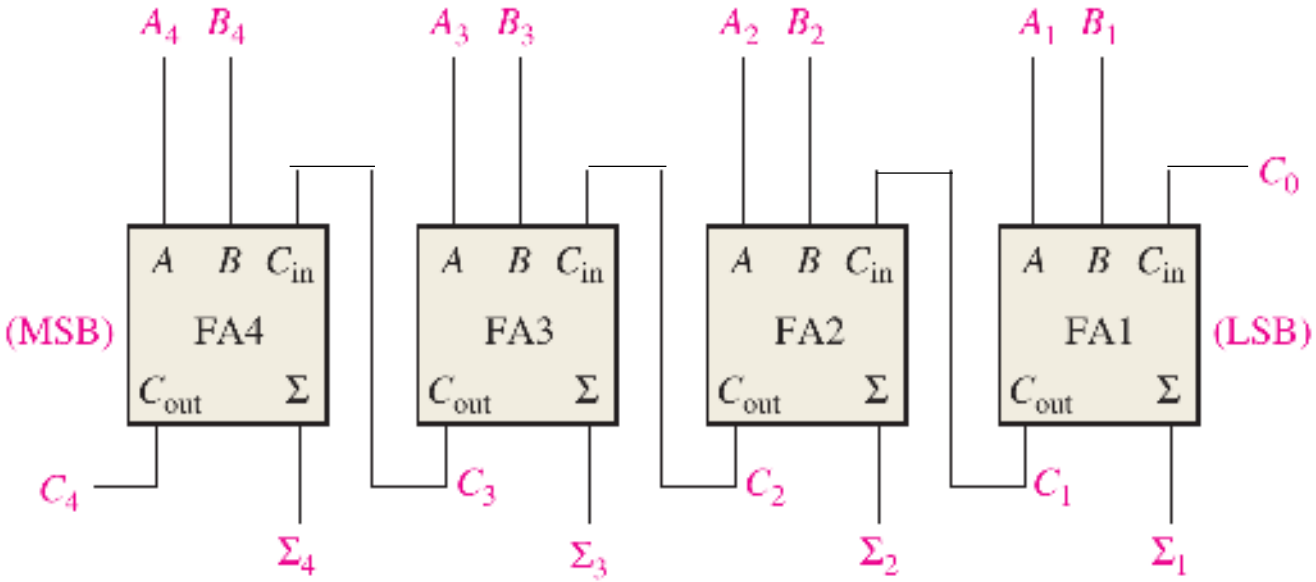
General format, addition of two 2-bit numbers:

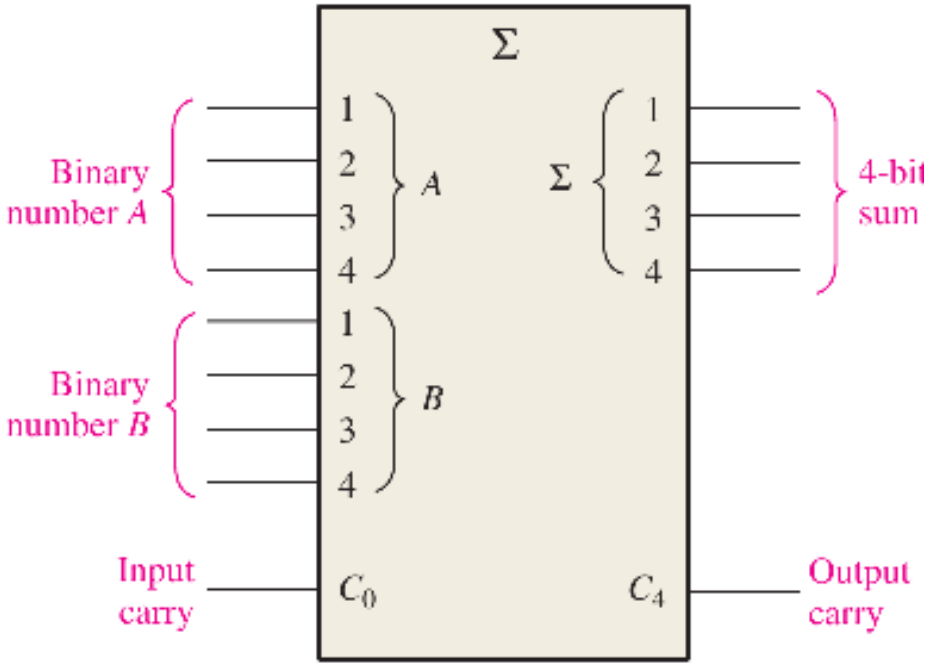
$$\begin{array}{r} A_2A_1 \\ +\ B_2B_1 \\ \hline \Sigma_3\Sigma_2\Sigma_1 \end{array}$$



n-bit binary adder

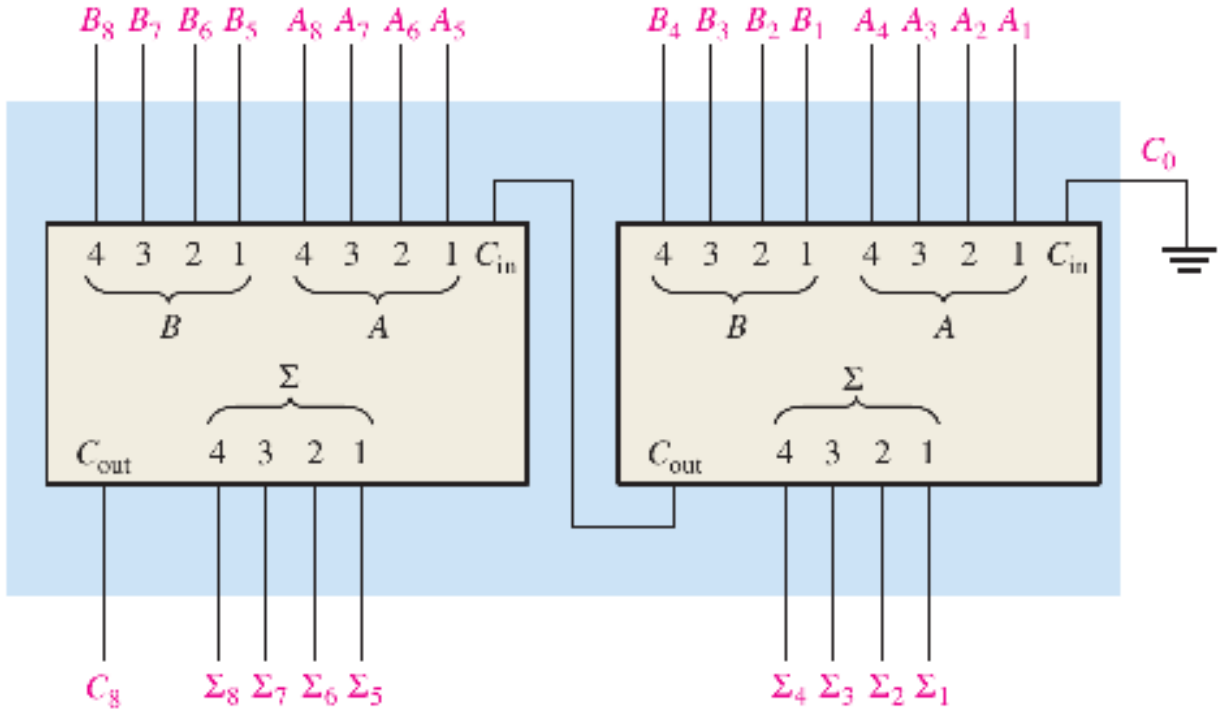
4-bit adder:





n-bit binary adder

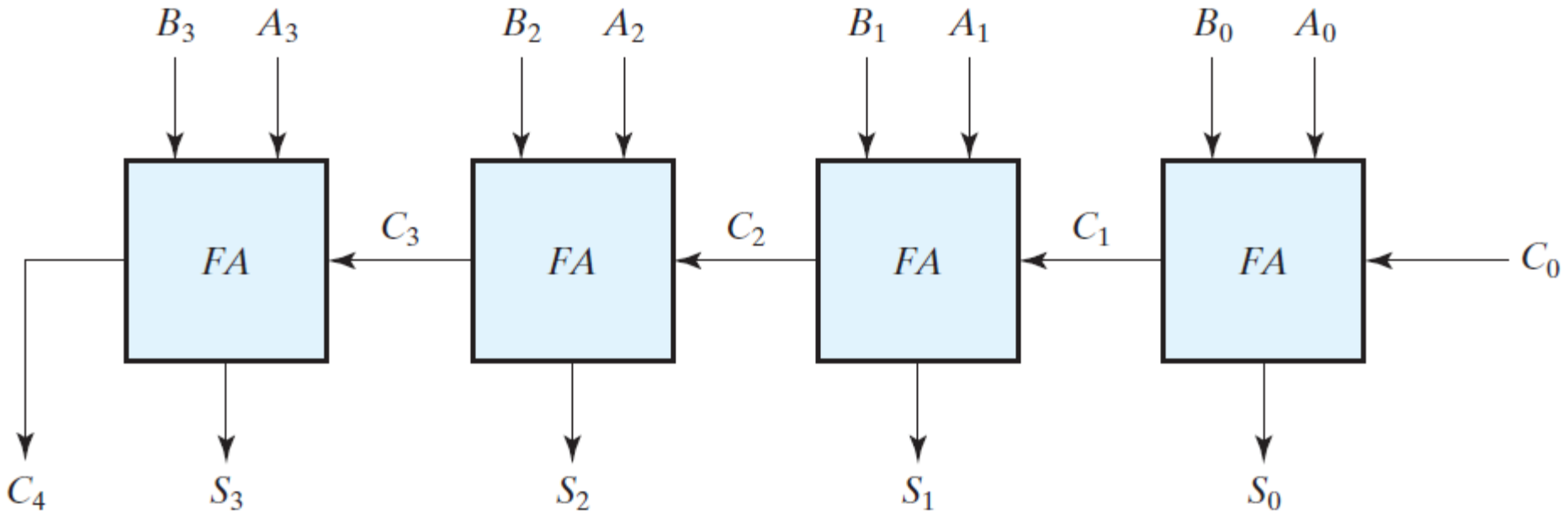
Adder expansion by cascading



Cascading of two 4-bit adders to form an 8-bit adder

n-bit binary adder

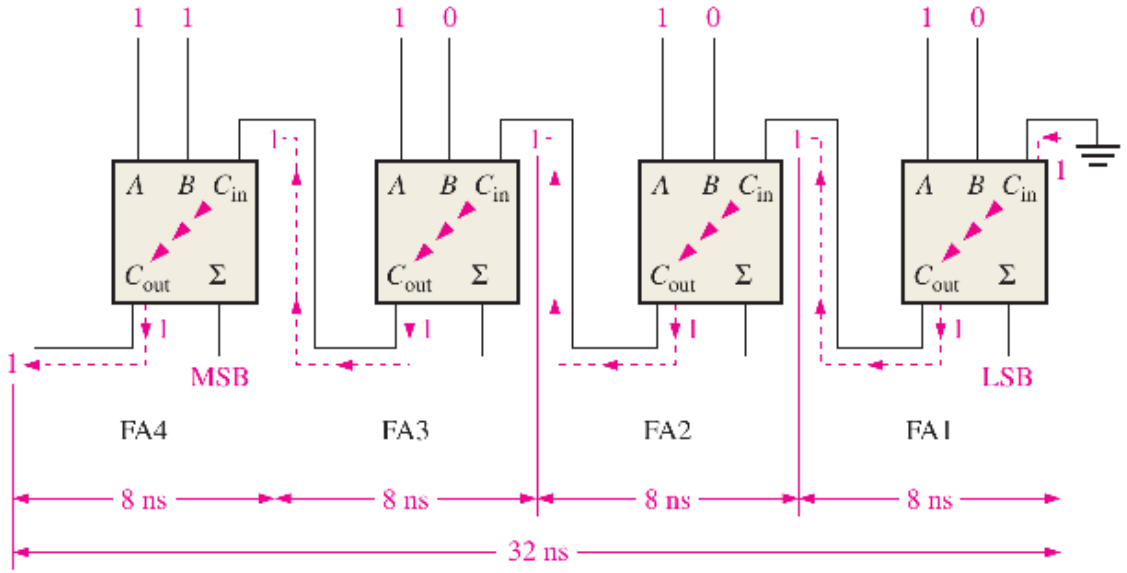
- Consider again the 4-bit adder: Can we make this circuit through the normal route?
- Note that the classical method would require a truth table (and K-map) with $2^9 = 512$ entries, since there are nine inputs to the circuit
- By using an iterative method of cascading a standard function, it is possible to obtain a simple and straightforward implementation



Ripple carry adder

Carry propagation problem

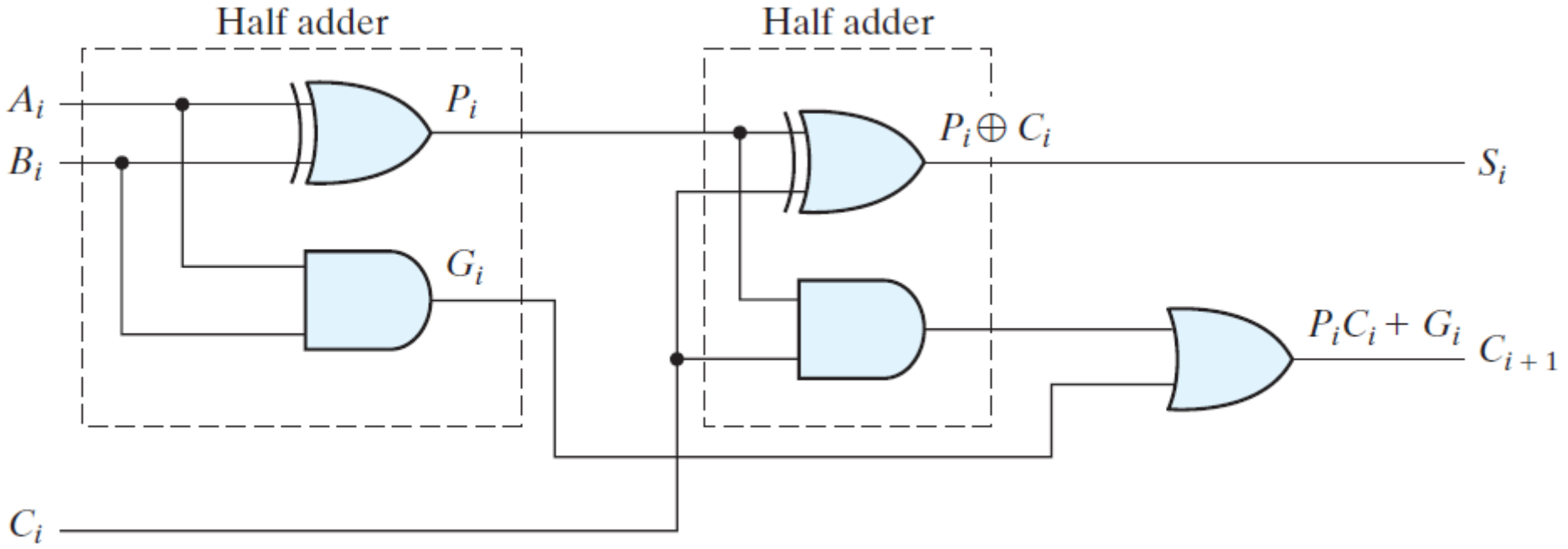
- The addition of two binary numbers in parallel implies that all the bits of the augend and addend are available for computation at the same time
- As in any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals
- The total propagation time is equal to the propagation delay of a typical gate, times the number of gate levels in the circuit
- The longest propagation delay time in an adder is the time it takes the carry to propagate through the full adders
- Since each bit of the sum output depends on the value of the input carry, the value of S_i at any given stage in the adder will be in its steady-state final value only after the input carry to that stage has been propagated



Ripple carry adder

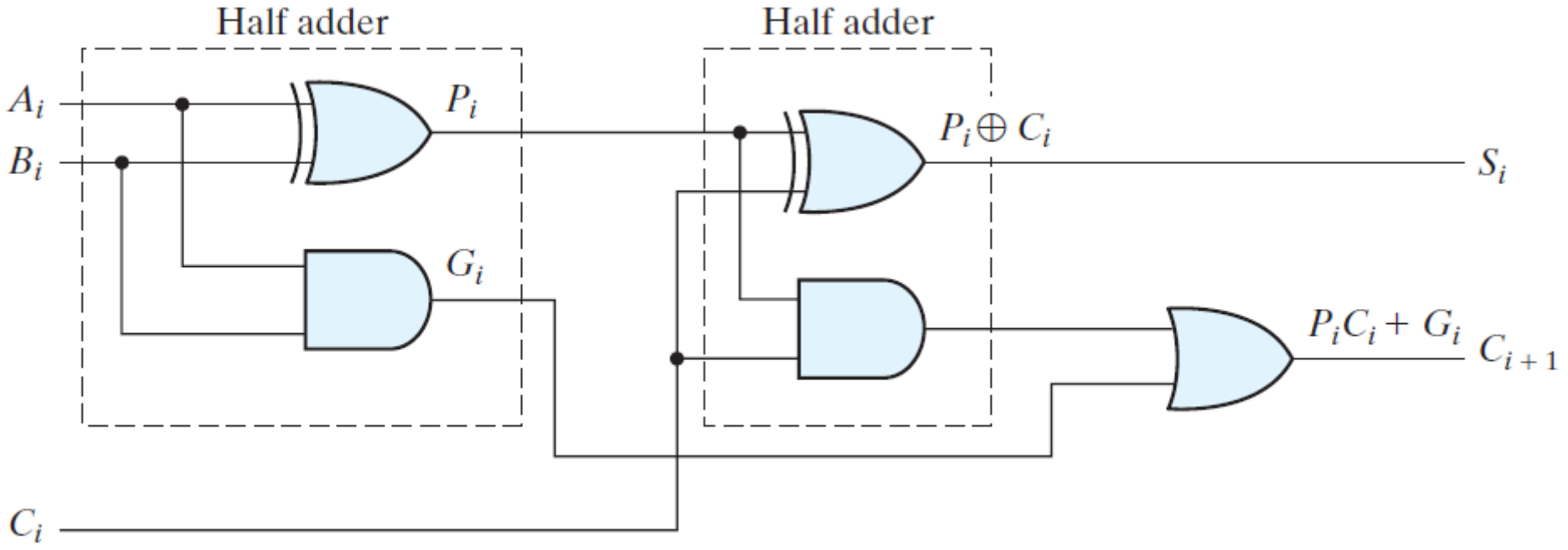
Carry propagation problem

- The number of gate levels for the carry propagation can be found from the circuit of the full adder
- The signals at P_i and G_i settle to their steady-state values after they propagate through their respective gates
- These two signals are common to all half adders and depend on only the input augend and addend bits
- The signal from the input carry C_i to the output carry C_{i+1} propagates through an AND gate and an OR gate, which constitute two gate levels
- If there are four full adders in the adder, the output carry C_4 would have 2 * 4 = 8 gate levels from C_0 to C_4
- For an *n* -bit adder, there are 2*n* gate levels for the carry to propagate from input to output



Carry propagation problem

- There are several techniques for reducing the carry propagation time in a parallel adder
- An obvious solution to this problem is to actually make the 2^n truth-table, K-map and get a two level implementation (either SoP or PoS)
- The most widely used technique employs the principle of *carry lookahead logic*

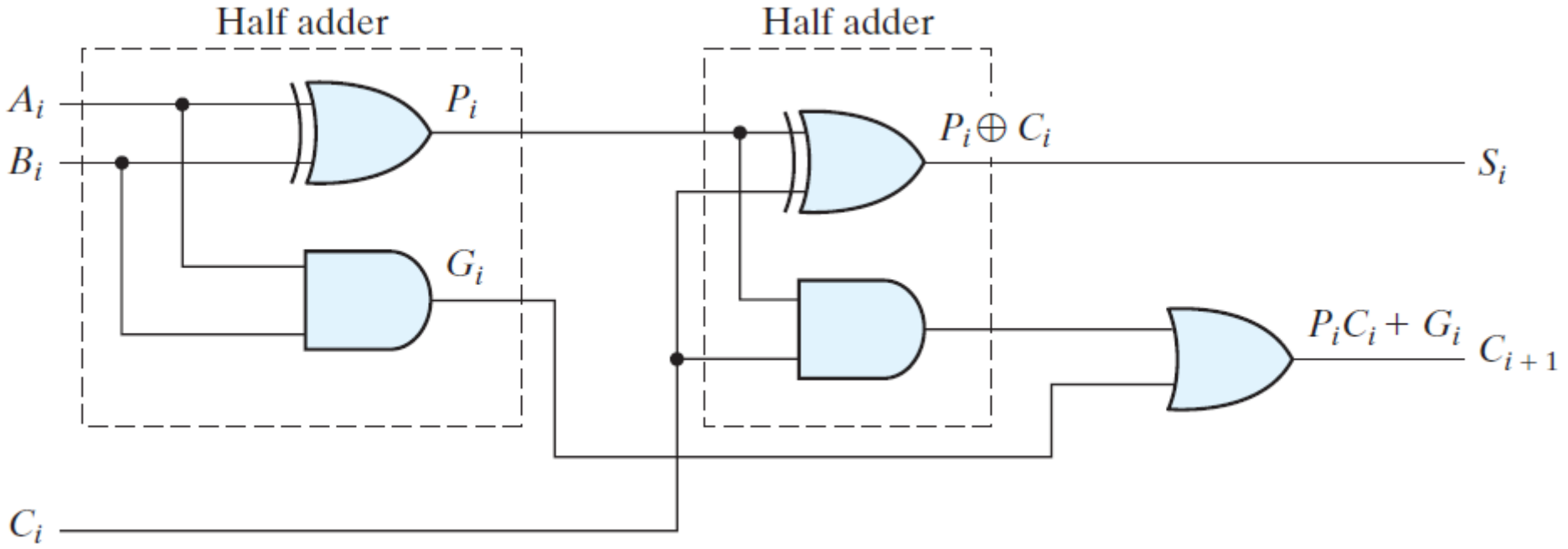


Carry propagation problem

- With the definition of P and G, we can write:

$$S_i = P_i \oplus C_i \text{ and } C_{i+1} = G_i + P_i C_i$$

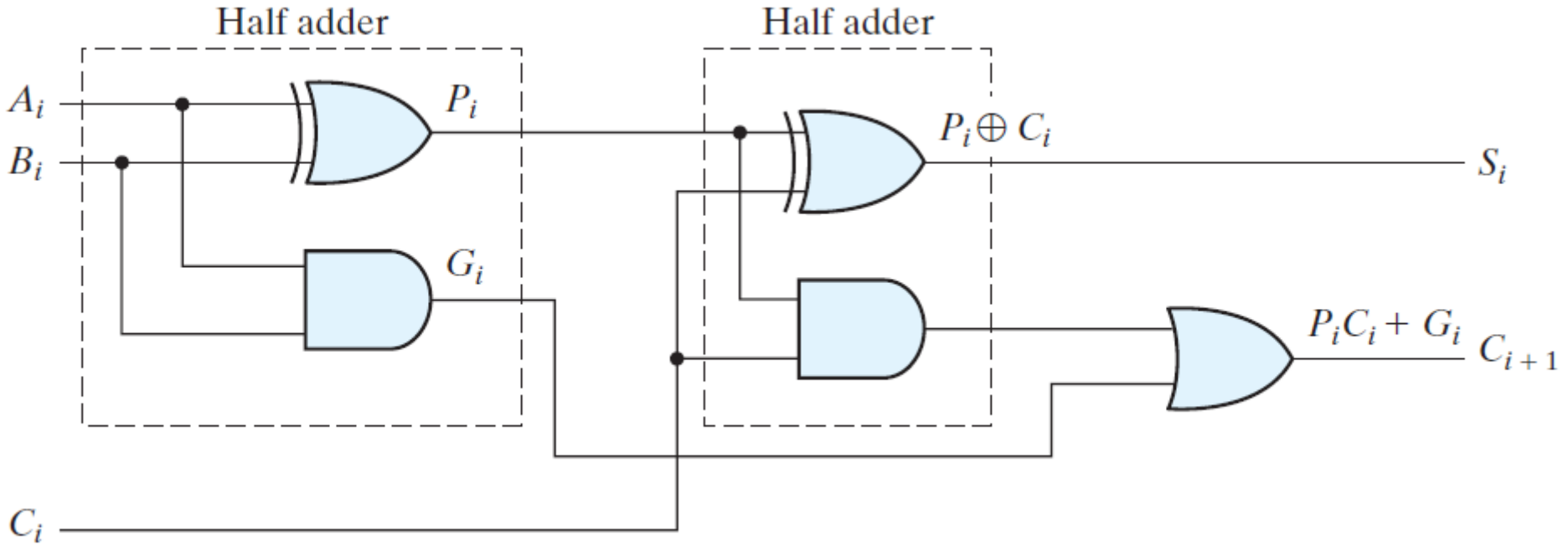
- G_i is called a *carry generate*, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i
- P_i is called a *carry propagate*, because it determines whether a carry into stage *i* will propagate into stage *i* + 1



Carry propagation problem

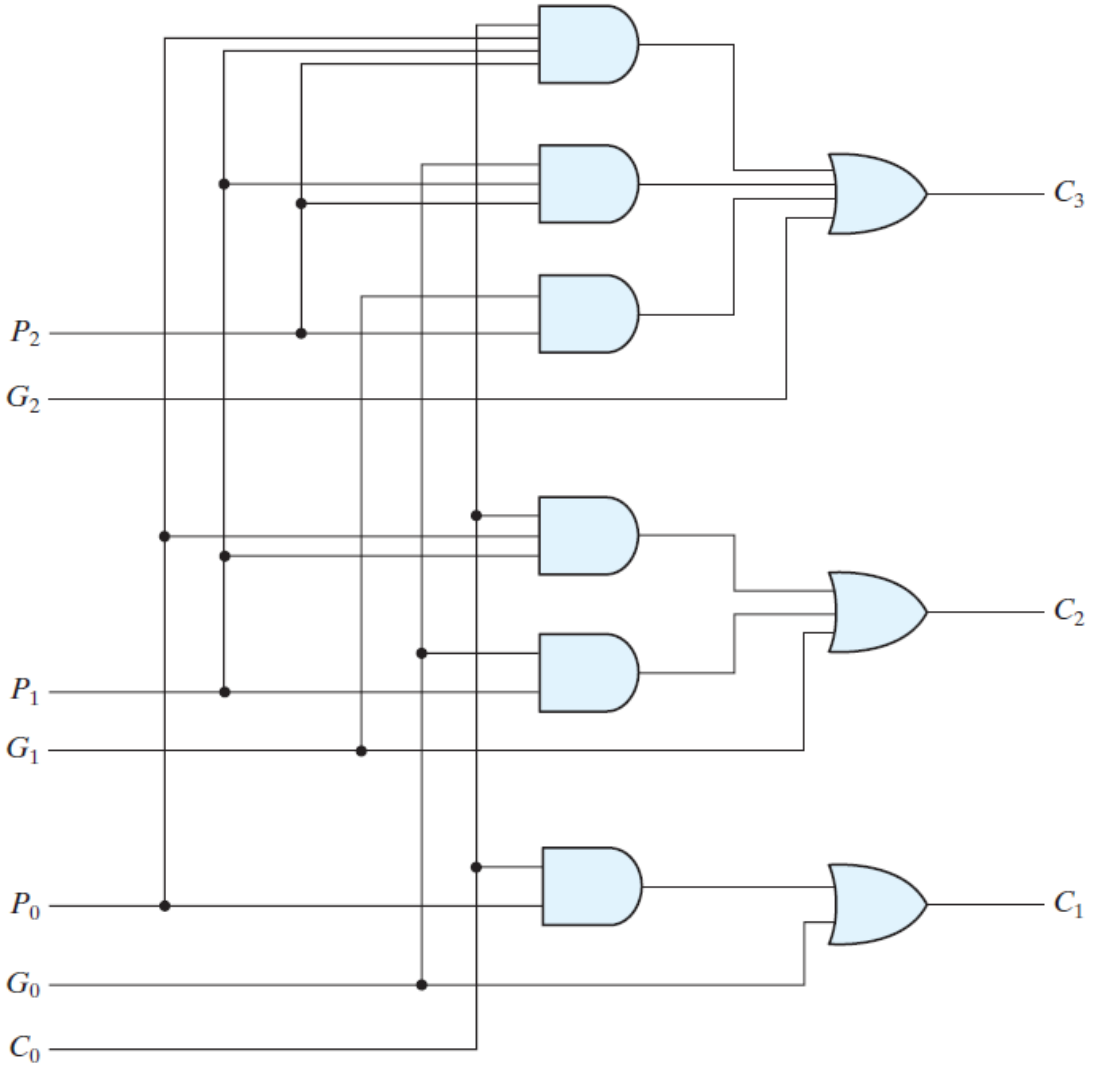
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations ($C_{i+1} = G_i + P_i C_i$):

$$C_0 = input\ carry$$
$$C_1 = G_0 + P_0 C_0$$
$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$
$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$



Carry propagation problem

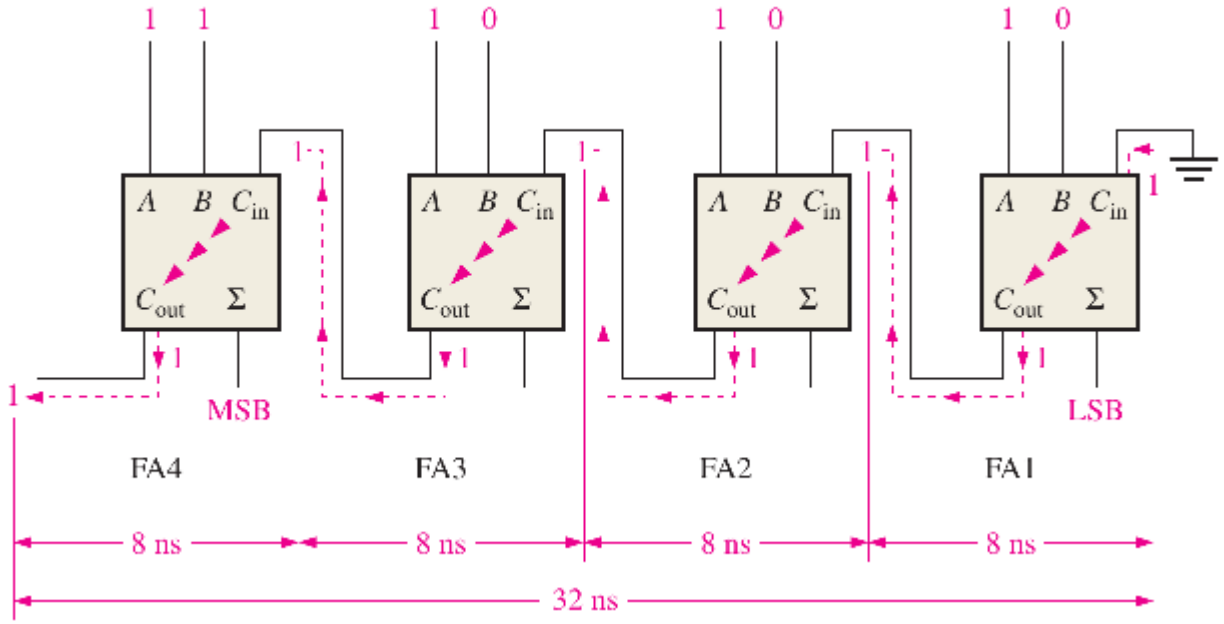
- Since the Boolean function for each output carry is expressed in sum-of-products form only dependent on P and G, each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND)
- Note that this circuit can add in less time because C_3 does not have to wait for C_2 and C_1 to propagate; in fact, C_3 is propagated at the same time as C_1 and C_2
- This gain in speed of operation is achieved at the expense of additional complexity (hardware)



Lecture 14 – Combinational circuits 3

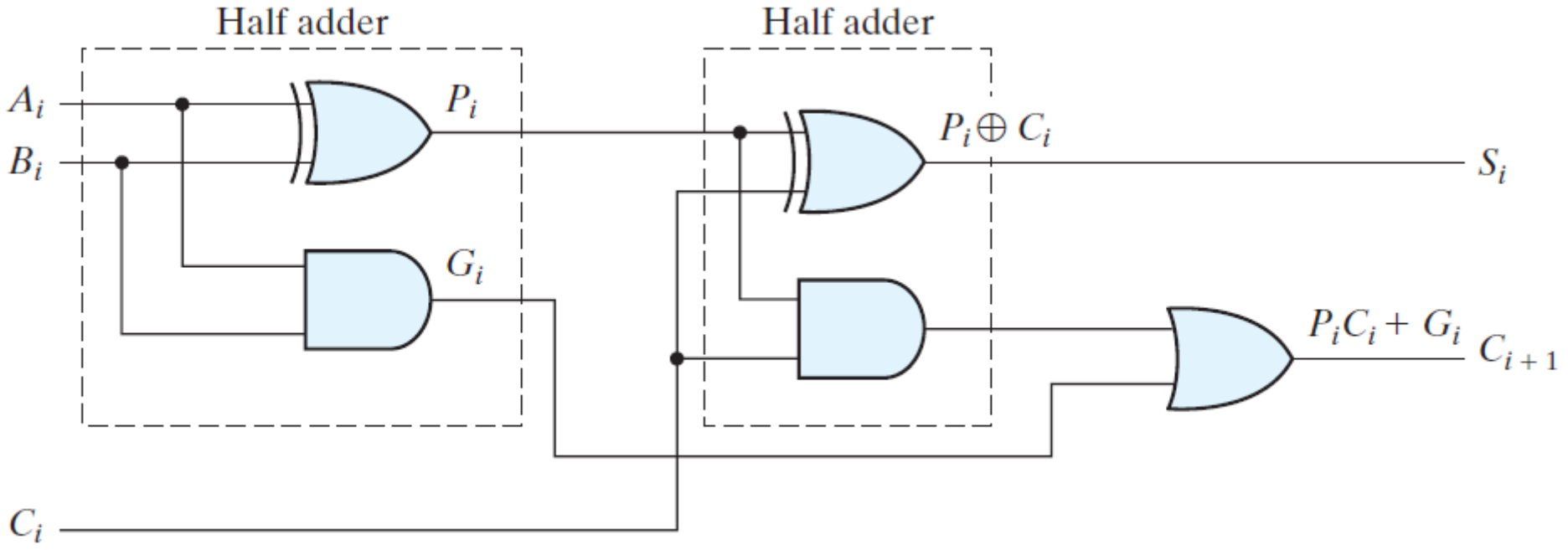
Carry propagation problem

- The addition of two binary numbers in parallel implies that all the bits of the augend and addend are available for computation at the same time
- As in any combinational circuit, the signal must propagate through the gates before the correct output sum is available in the output terminals
- The total propagation time is equal to the propagation delay of a typical gate, times the number of gate levels in the circuit



Carry propagation problem

- There are several techniques for reducing the carry propagation time in a parallel adder
- An obvious solution to this problem is to actually make the 2^n truth-table, K-map and get a two level implementation (either SoP or PoS)
- The most widely used technique employs the principle of *carry lookahead logic*

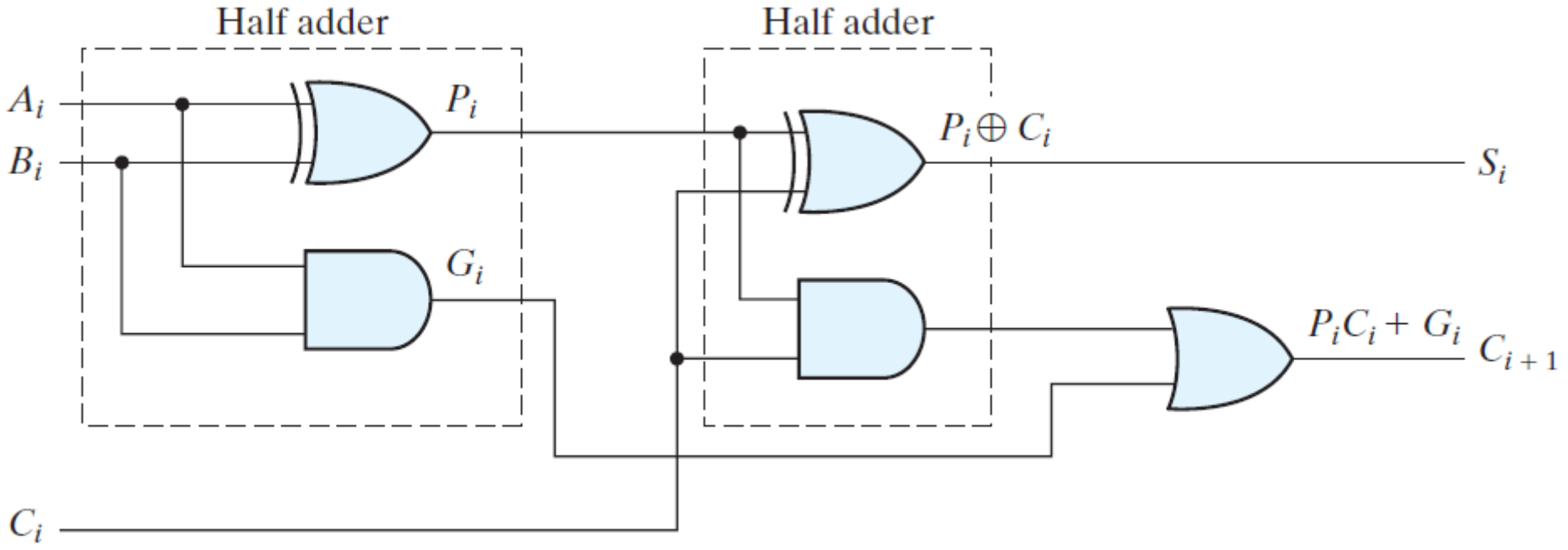


Carry propagation problem

- With the definition of P and G, we can write:

$$S_i = P_i + C_i \text{ and } C_{i+1} = G_i + P_i C_i$$

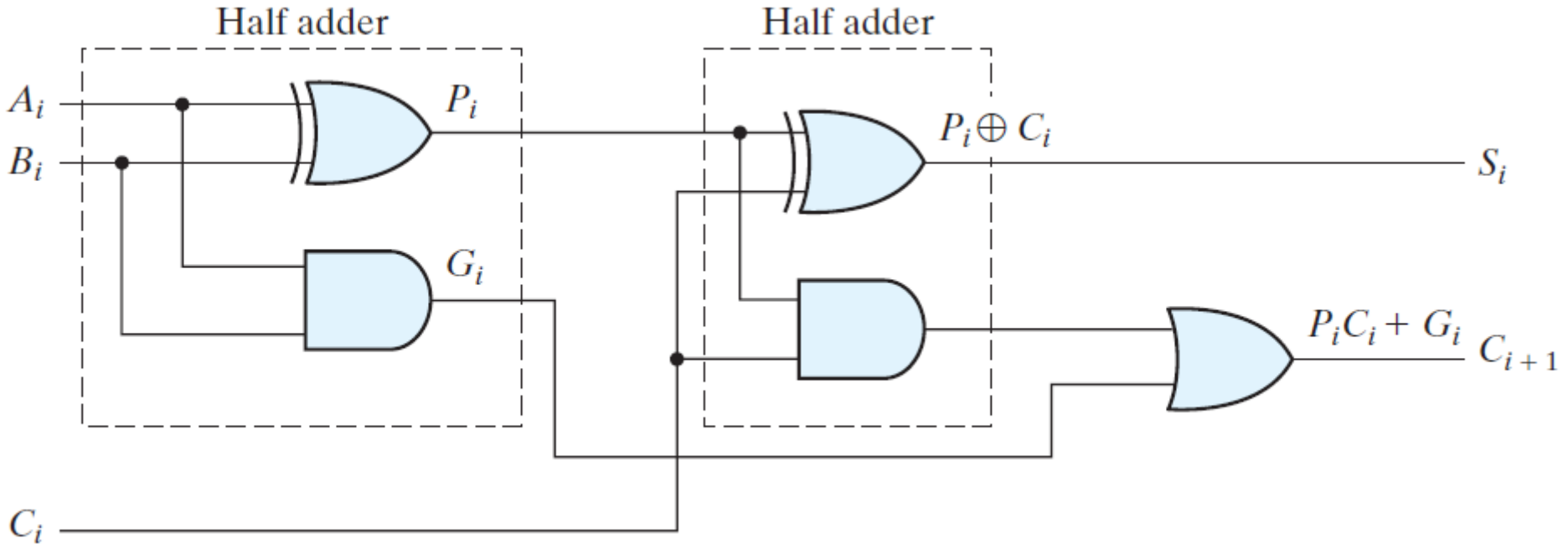
- G_i is called a *carry generate*, and it produces a carry of 1 when both A_i and B_i are 1, regardless of the input carry C_i
- P_i is called a *carry propagate*, because it determines whether a carry into stage *i* will propagate into stage *i* + 1



Carry propagation problem

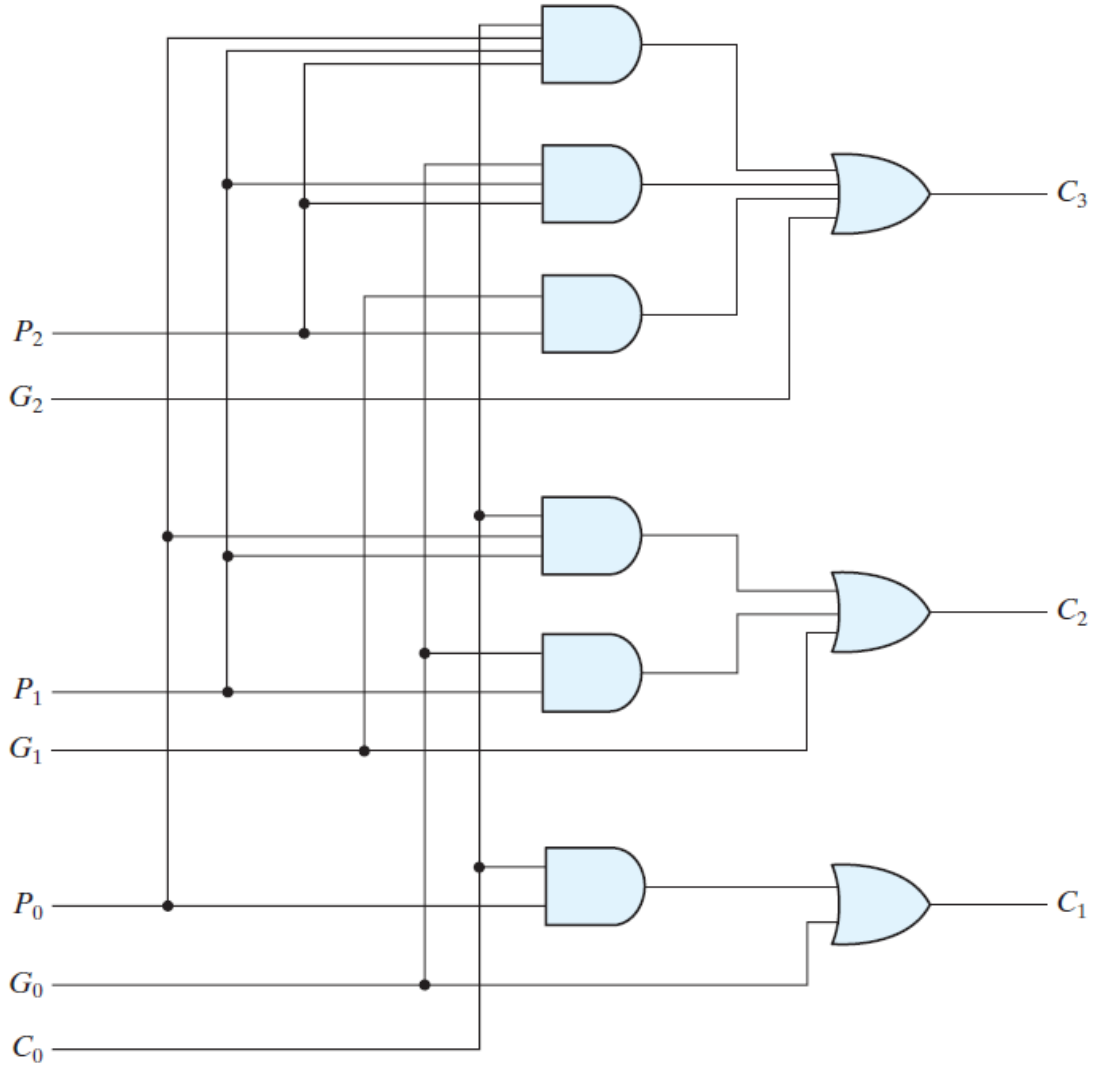
- We now write the Boolean functions for the carry outputs of each stage and substitute the value of each C_i from the previous equations:

$$C_0 = input\ carry$$
$$C_1 = G_0 + P_0C_0$$
$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$$
$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$



Carry propagation problem

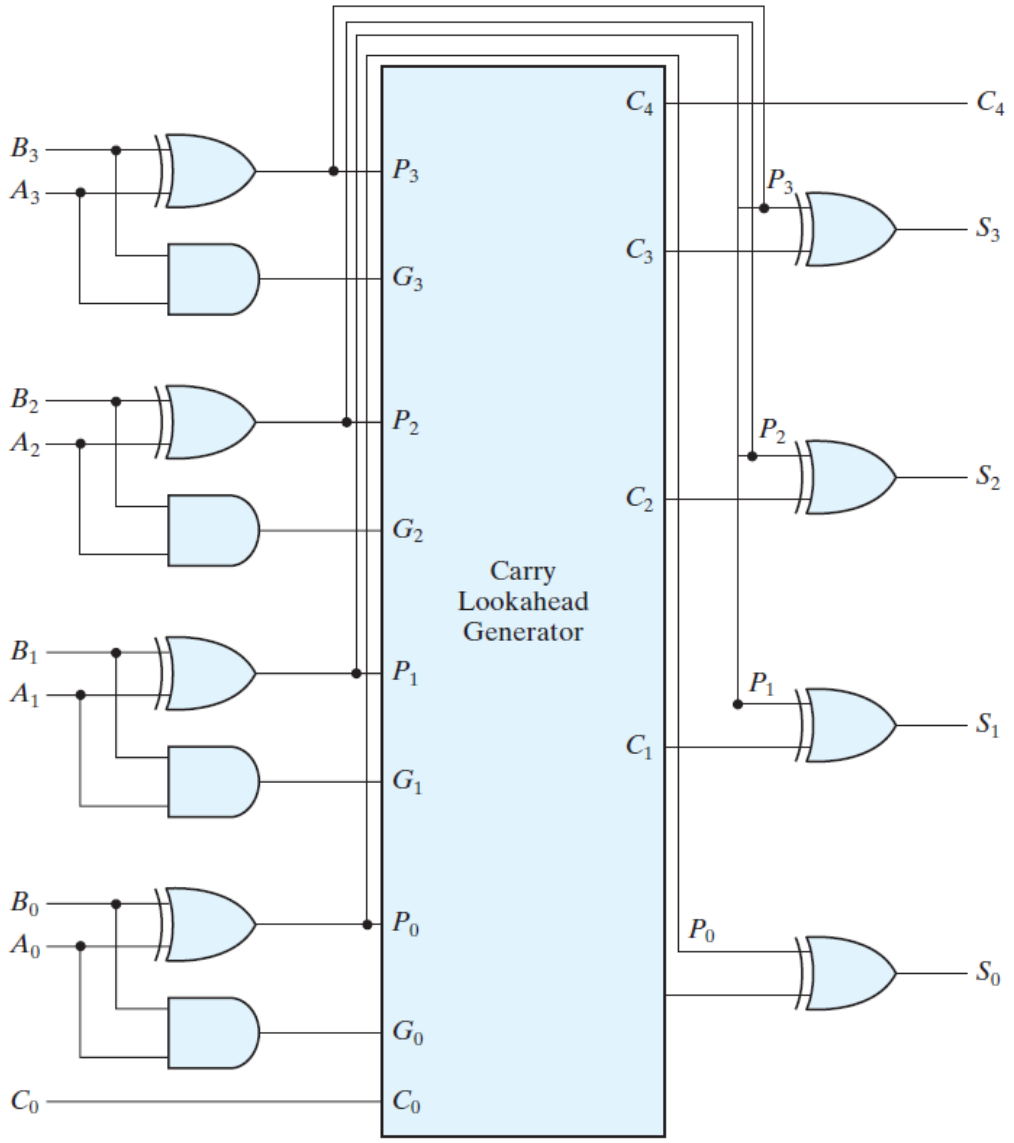
- Each output carry is expressed in sum-of-products form only dependent on P and G, each function can be implemented with one level of AND gates followed by an OR gate
- This circuit can add in less time because C_3 does not have to wait for C_2 and C_1 to propagate; in fact, C_3 is propagated at the same time as C_1 and C_2
- This gain in speed of operation is achieved at the expense of additional complexity (hardware)



Carry lookahead generator

Carry propagation problem

- Each sum output requires two XOR gates
- The output of the first XOR gate generates the P_i variable, and the AND gate generates the G_i variable
- The carries are propagated through the carry lookahead generator and applied as inputs to the second XOR gate
- *All output carries are generated after a delay through only two levels of gates*
- Thus, outputs S_1 through S_3 have equal propagation delay times



The Binary subtractor

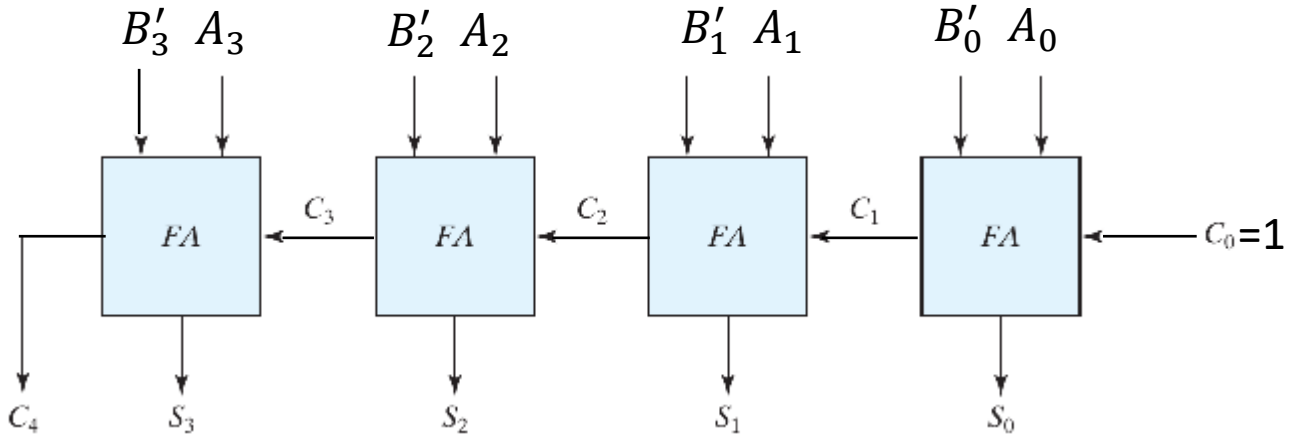
Revisit : Subtraction with Radix complements

- Subtraction using method of borrowing is less efficient when implemented with digital hardware
- Consider subtraction $M - N$ in base *r*

- Here is the algorithm using Radix complement:
  1. Take radix complement of subtrahend *N*: $r^n - N$
  2. **Add** this to *M*: $(r^n - N) + M = r^n + (M - N) = r^n - (N - M)$
  3. If you get a carry in the $(n+1)^{th}$ digit, then the result is positive, discard the carry and you are done
  4. If you ***do not*** get a carry in the $(n+1)^{th}$ digit, then the result is **negative**. Take the radix complement of the number to get the answer, then put a negative sign

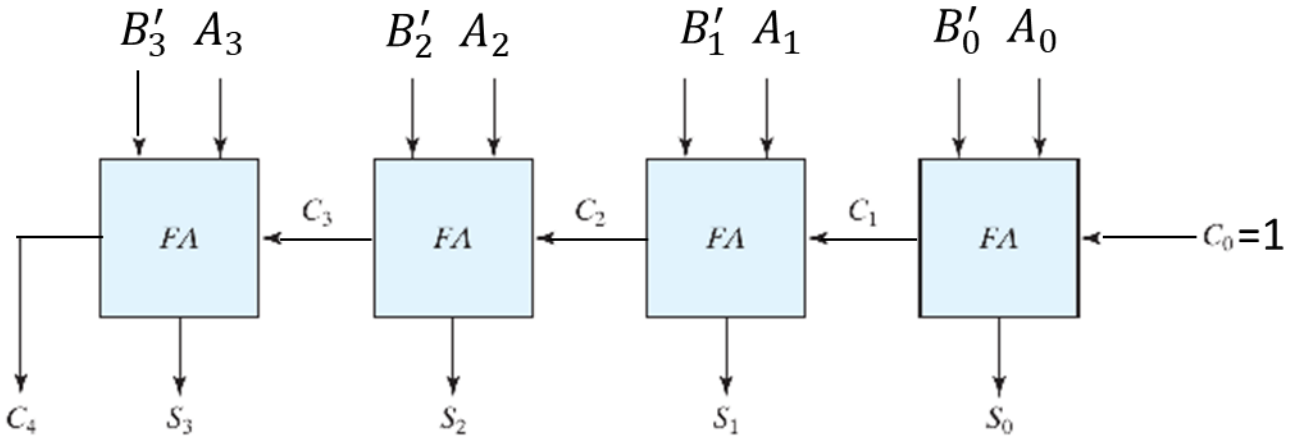
Binary subtractor

- The subtraction of binary numbers can be done most conveniently by means of complements
- Remember that the subtraction *A* - *B* can be done by taking the 2's complement of *B* and **adding** it to *A*
- The 2's complement can be obtained by taking the 1's complement and adding 1 to the least significant pair of bits
- The 1's complement can be implemented with inverters, and a 1 can be added to the sum through the input carry



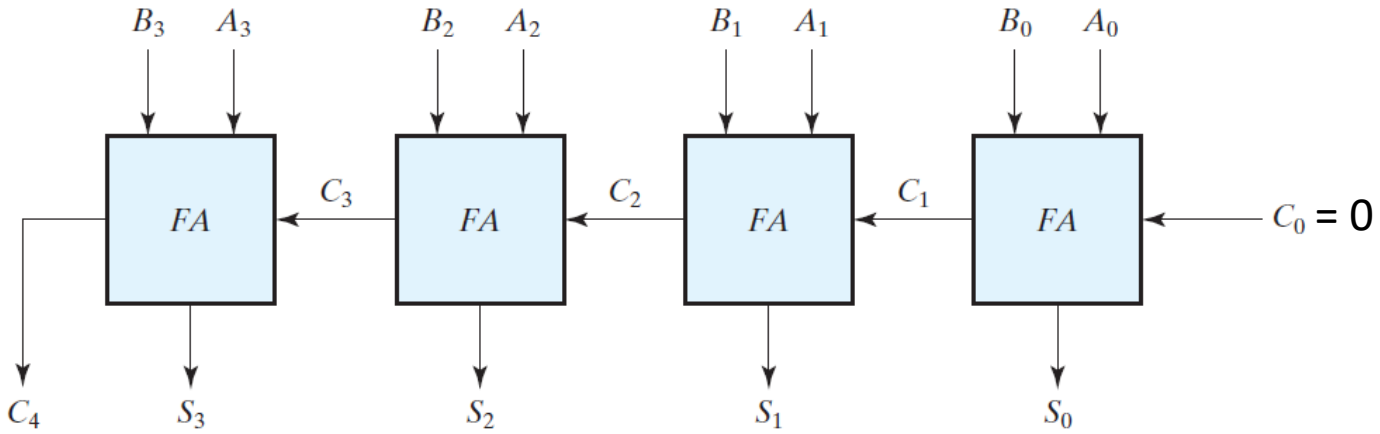
Binary subtractor

- The circuit for subtracting *A* - *B* consists of an adder with inverters placed between each data input *B* and the corresponding input of the full adder
- **The input carry C_0 must be equal to 1 when subtraction is performed**
- The operation thus performed becomes: *A + (1's complement of B) + 1*.
  - This is equal to *A + 2's complement of B*
- That gives *A* - *B* if $A \geq B$ or the 2's complement of *B* - *A* if $A < B$

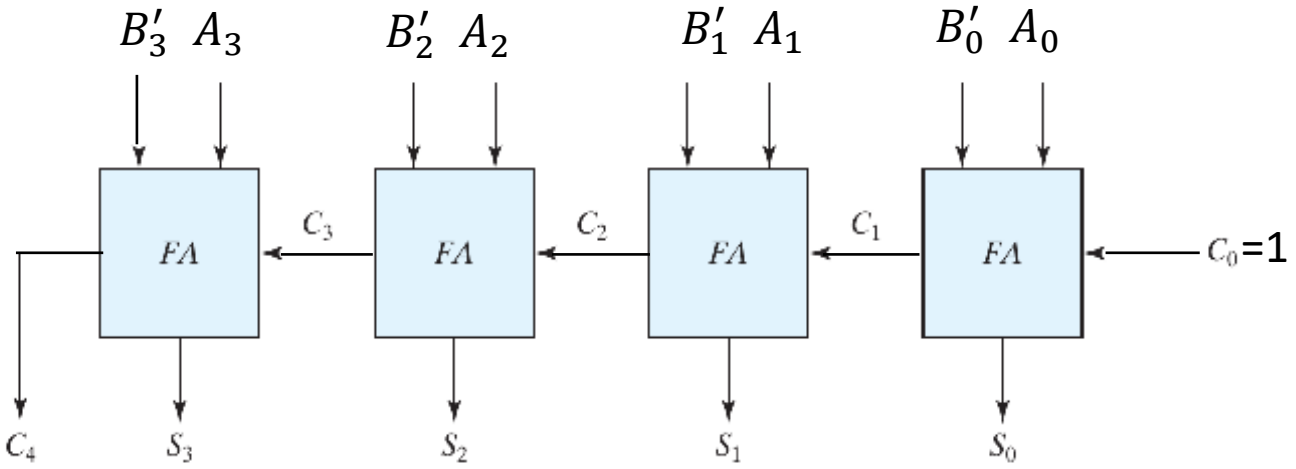


Can we combine the binary adder & subtractor

- Both use the 4-bit full adder. In one case, we use B and in another we use inverted B



Adder

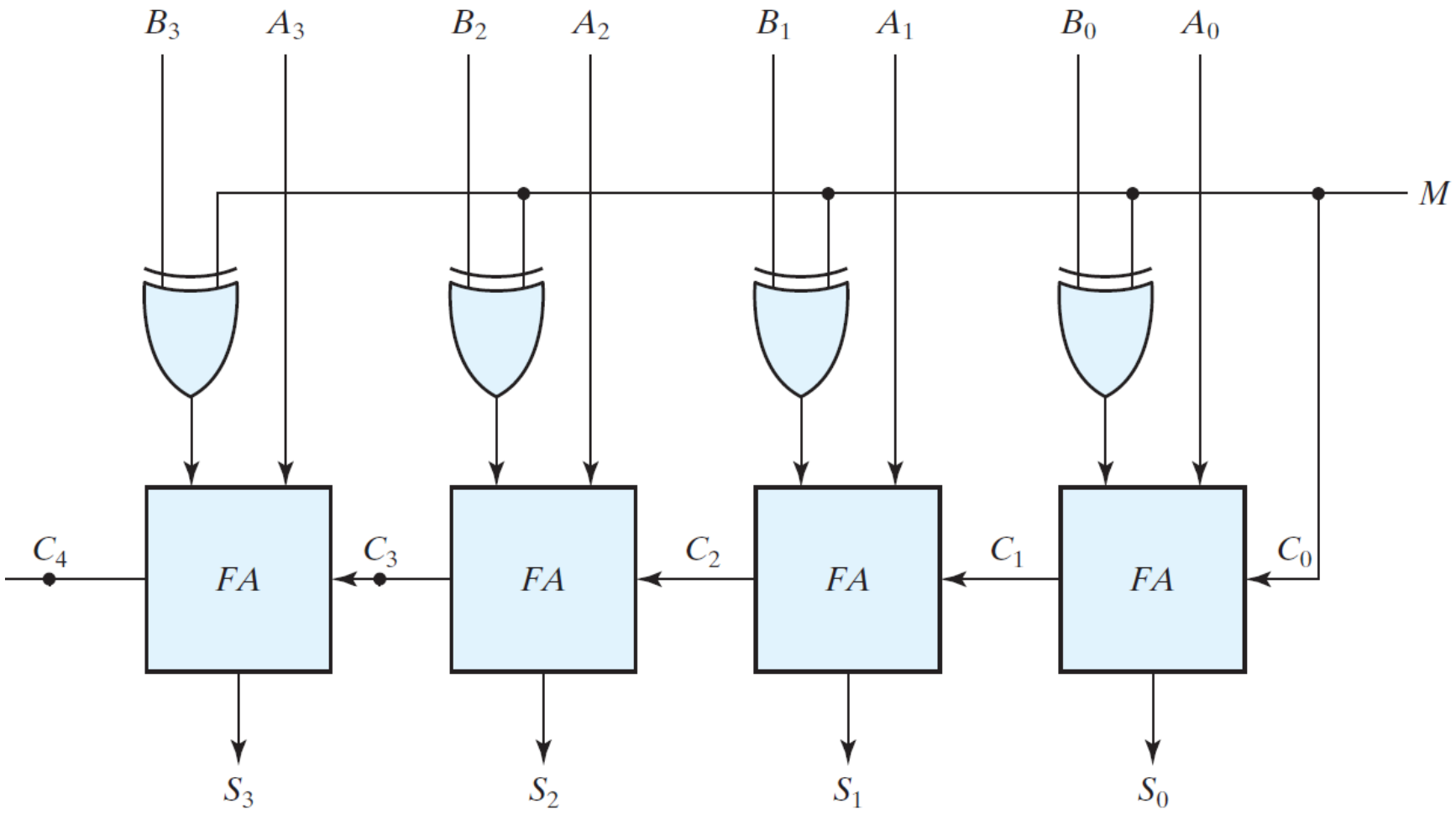


Subtractor

Binary adder-subtractor

- Here is some magic: The addition and subtraction operations can be combined into one circuit
- The mode input *M* controls the operation
- When *M* = 0, the circuit is an adder, and when *M* = 1, the circuit becomes a subtractor
- When *M* = 0, the full adders receive the value of *B* , the input carry is 0, and the circuit performs *A* + *B*
- When *M* = 1, the full adders receive *B'* and $C_0 = 1$
- Thus, the *B* inputs are all complemented and a 1 is added through the input carry
- The circuit performs the operation *A* plus the 2's complement of *B*

Binary adder-subtractor



The BCD adder

BCD adder

- Consider the arithmetic addition of two decimal digits in BCD, together with an input carry from a previous stage
- Since each input digit does not exceed 9, the output sum cannot be greater than 9 + 9 + 1 = 19, the 1 in the sum being an input carry
- Suppose we apply two BCD digits to a four-bit binary adder
- The adder will form the sum in *binary* and produce a result that ranges from 0 through 19

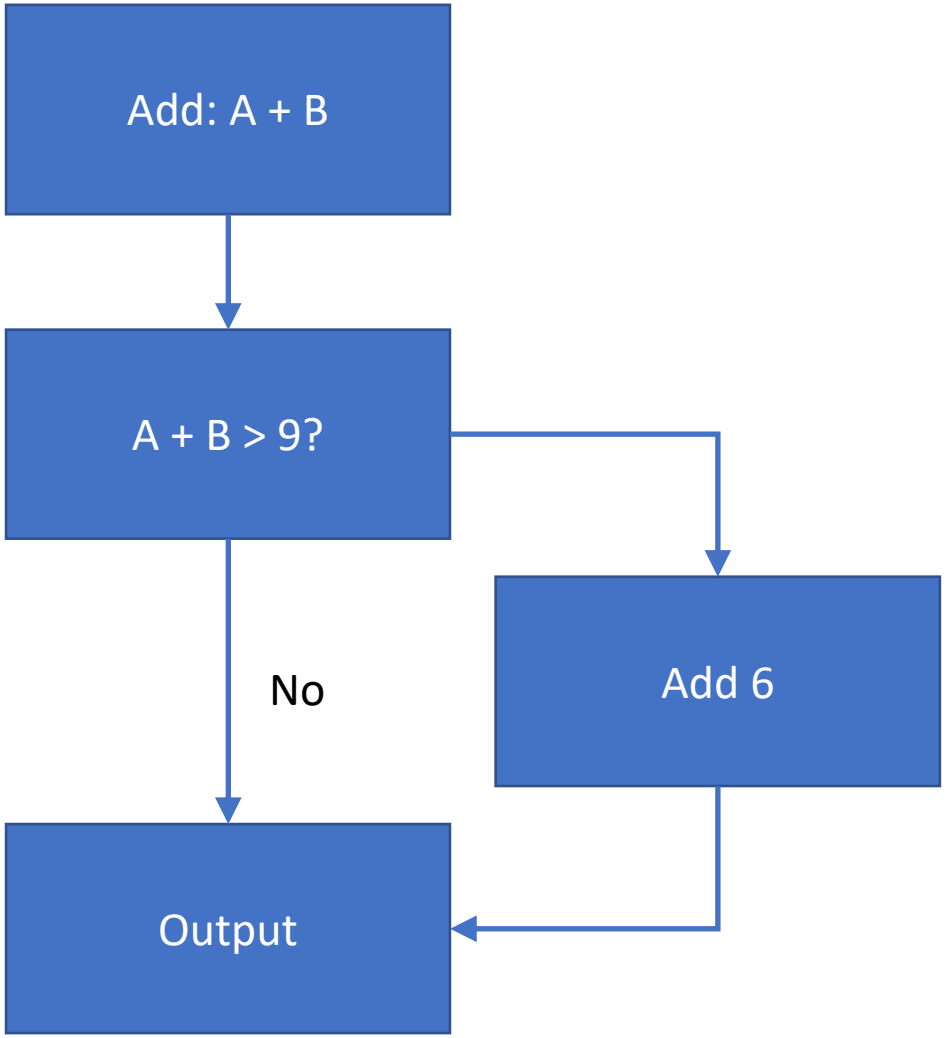
Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

BCD adder

- When the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed
- When the binary sum is greater than 1001, we obtain an invalid BCD representation
- *The addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and also produces an output carry as required*

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

BCD adder - algorithm



Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

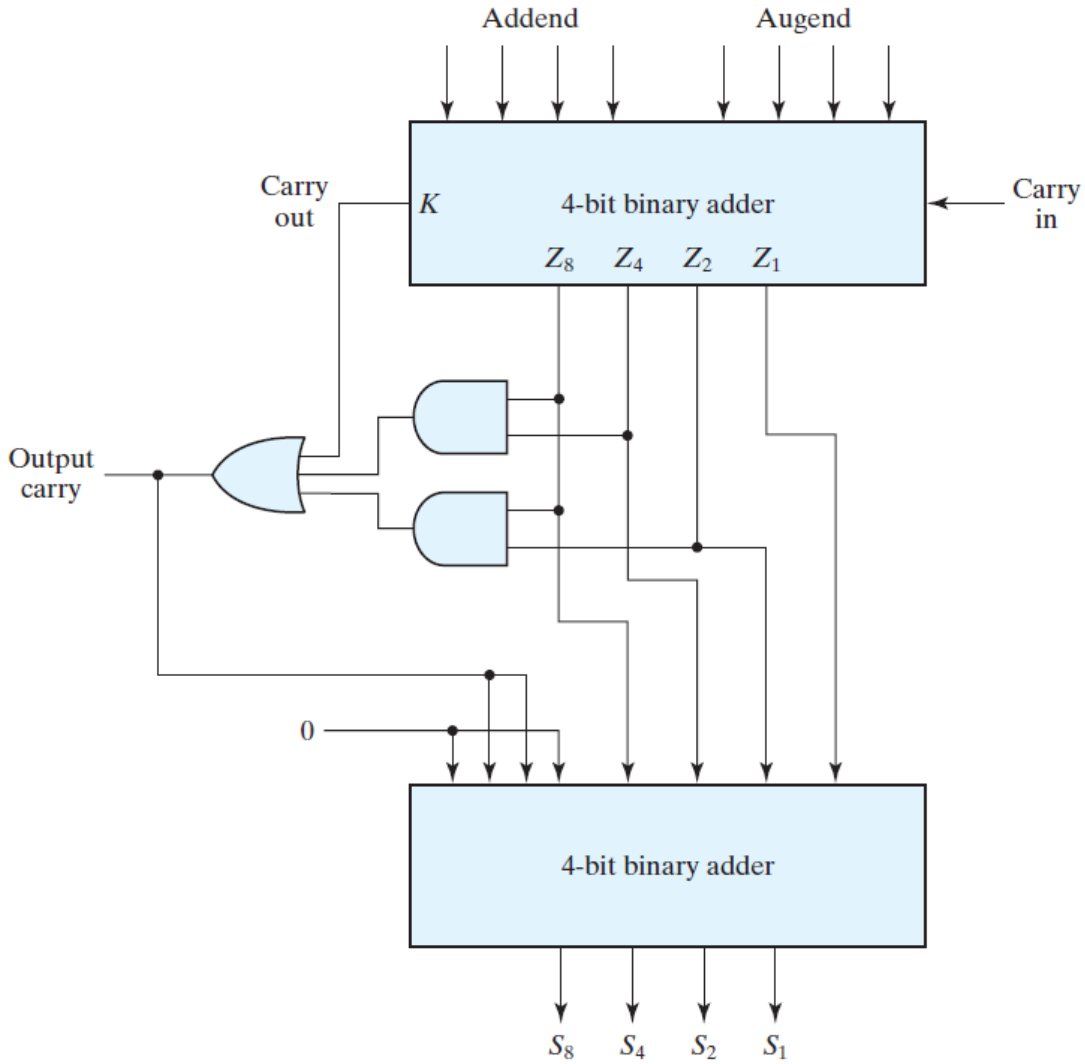
BCD adder

- It is obvious that a correction is needed when the binary sum has an output carry $K = 1$
- The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z_8
- To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_8, we specify further that either Z_4 or Z_2 must have a 1
- Thus, the condition for a correction and an output carry can be expressed by the Boolean function: $Cor = K + Z_8Z_4 + Z_8Z_2$
- When *Cor* = 1, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

BCD adder

- The logic circuit that detects the necessary correction can be derived from the entries in the table
- It is obvious that a correction is needed when the binary sum has an output carry $K = 1$
- The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z_8
- To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_8, we specify further that either Z_4 or Z_2 must have a 1
- Thus, the condition for a correction and an output carry can be expressed by the Boolean function: $Cor = K + Z_8Z_4 + Z_8Z_2$
- When *Cor* = 1, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage



The Binary multiplier

Binary multiplier

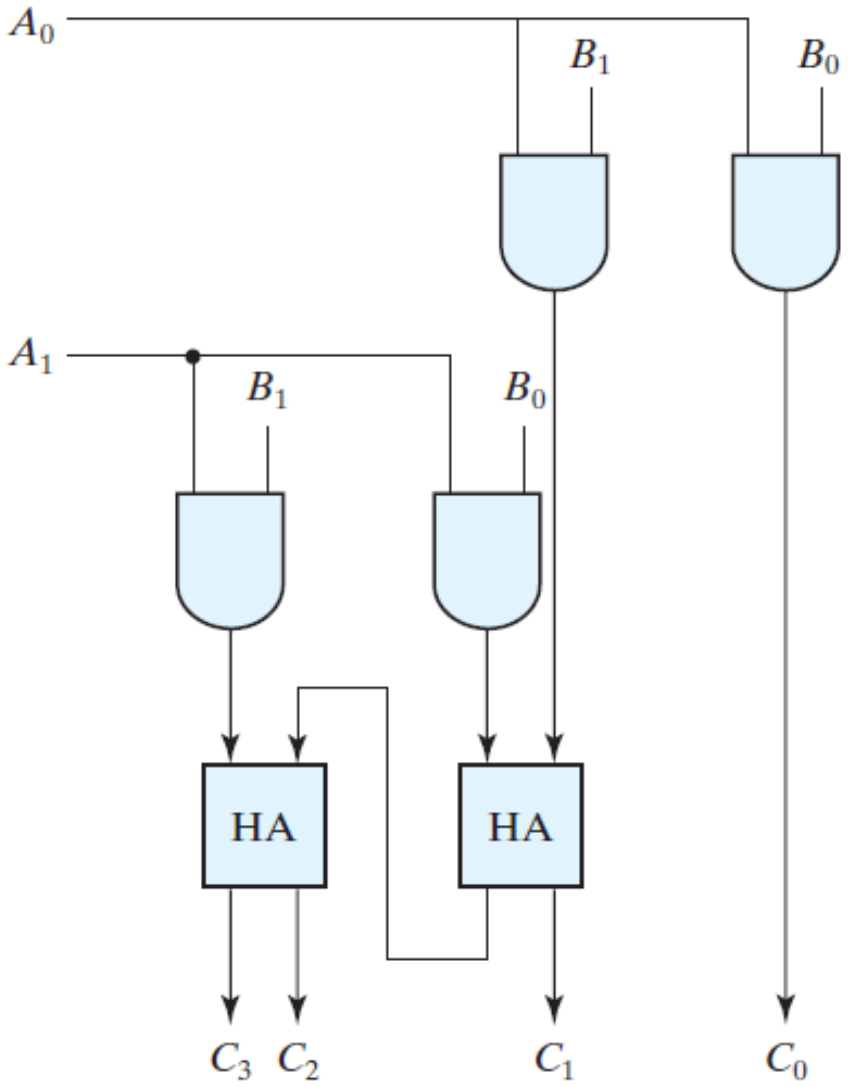
- Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers
- The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit
- Each such multiplication forms a partial product
- Successive partial products are shifted one position to the left
- The final product is obtained from the sum of the partial products
- Consider a 2-bit x 2-bit multiplier. Number of inputs/outputs?

$$
\begin{array}{cccc}
 & & B_1 & B_0 \\
 & & A_1 & A_0 \\
\hline
 & & A_0B_1 & A_0B_0 \\
 & A_1B_1 & A_1B_0 & \\
\hline
C_3 & C_2 & C_1 & C_0
\end{array}
$$

Binary multiplier

- The multiplicand bits are B_1 and B_0, the multiplier bits are A_1 and A_0, and the product is $C_3C_2C_1C_0$

$$
\begin{array}{cccc}
 & & B_1 & B_0 \\
 & & A_1 & A_0 \\
\hline
 & & A_0B_1 & A_0B_0 \\
 & A_1B_1 & A_1B_0 & \\
\hline
C_3 & C_2 & C_1 & C_0
\end{array}
$$



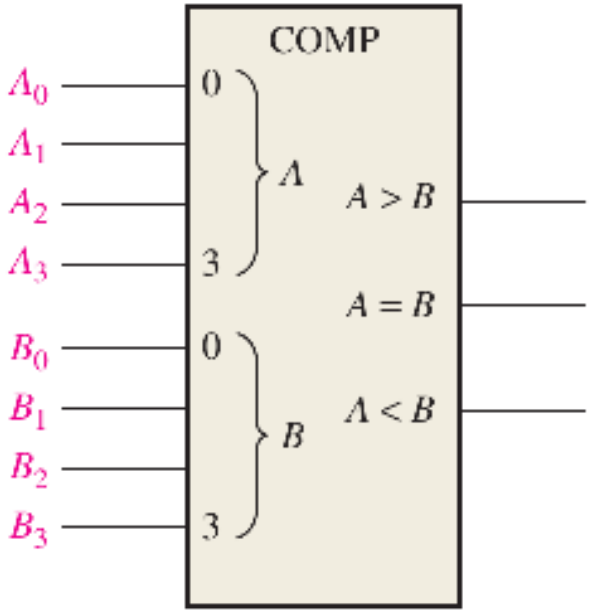
Lecture 15 – Combinational circuits 4

Chapter 5

The Binary comparator

Binary comparator

- The comparison of two numbers is an operation that determines whether one number is greater than, less than, or equal to the other number
- A *magnitude comparator* is a combinational circuit that compares two numbers *A* and *B* and determines their relative magnitudes
- The outcome of the comparison is specified by three binary variables that indicate whether $A > B$, $A = B$, or $A < B$
- On the one hand, the circuit for comparing two *n*-bit numbers has 2^{2n} entries in the truth table and becomes too cumbersome, even with $n = 3$
- On the other hand, as one may suspect, a comparator circuit possesses a certain amount of regularity



Binary comparator

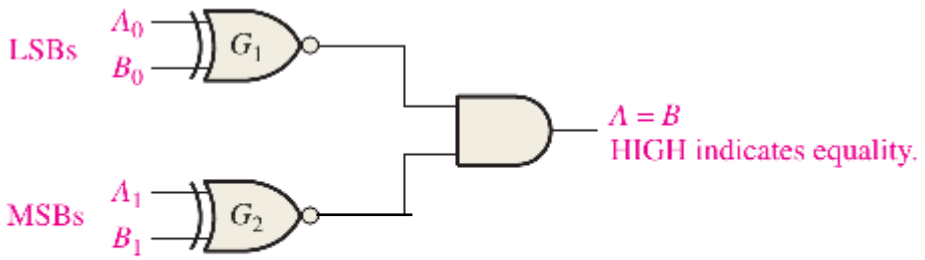
- Consider two numbers, *A* and *B* , with four digits each $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$
- The two numbers are equal if all pairs of significant digits are equal: $A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$, **and** $A_0 = B_0$
- To check bit-wise equality, we can use the XNOR gate

$$x_i = A_iB_i + A_i'B_i' \text{ for } i = 0, 1, 2, 3$$

- For equality to exist, all x_i variables must be equal to 1, a condition that dictates an AND operation of all variables:

$$(A = B) = x_3x_2x_1x_0$$

Eg: Comparison of 2-bit numbers



General format: Binary number $A \rightarrow A_1A_0$
Binary number $B \rightarrow B_1B_0$

Binary comparator

- To determine whether *A* is greater or less than *B*, we inspect the relative magnitudes of pairs of significant digits, starting from the most significant position
- If the two digits of a pair are equal, we compare the next lower significant pair of digits
- The comparison continues until a pair of unequal digits is reached
- If the corresponding digit of *A* is 1 and that of *B* is 0, we conclude that $A > B$. Else, we have $A < B$

Eg: $A_3 A_2 A_1 A_0 = 1001$ and $B_3 B_2 B_1 B_0 = 1010$

Binary comparator

- The comparison can be expressed logically by the two Boolean functions:

$$(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$$(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$$

Binary comparator

$x_i = A_i B_i + A_i' B_i'$ for $i = 0, 1, 2, 3$

$(A = B) = x_3 x_2 x_1 x_0$

$(A > B)$
$= A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1'$
$+ x_3 x_2 x_1 A_0 B_0'$

$(A < B)$
$= A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1$
$+ x_3 x_2 x_1 A_0' B_0$

Interesting: Can we prove that only one of (A=B), (A>B) and (A<B) will be “1” at any given time?

